Dynamic Features for Robust Malware Detection: A Systematic Review, Taxonomy, and Practical Analysis Framework

Monday Onoja [a,*], Peter Anthony[a], Zekeri Adams[a], Kefas Rimamnuskeb Galadima[b], Martin Homola[a], Štefan Balogh[b], Ján Kľuka[a], Roderik Ploszek[b], Temidayo Oluwatosin Omotehinwa[c], Abayomi Joshua Jegede[d]

[a] *Department of Applied Informatics, Faculty of Mathematics, Physics and Informatics, Comenius University, Mlynská dolina, Bratislava, Slovakia*
[b] *Institute of Computer Science and Mathematics, Faculty of Electrical Engineering and Information Technology, Slovak University of Technology, Ilkovičova 3, Bratislava, Slovakia*
[c] *Department of Mathematics and Computer Science, Federal University of Health Sciences, , Otukpo, Benue State, Nigeria*
[d] *Department of Computer Sciences, Edo State University, Km7, Auchi-Abuja Road, Iyamho-Uzairue, Edo State , Nigeria*

Abstract

The need to mitigate malware attacks cannot be overemphasized, as they pose serious threats to the critical information assets in the cyberspace. Understanding and utilizing appropriate malware features is a game changer for implementing a high-performance malware detection system. Static analysis has been applied to malware features, but it has limitations in detecting obfuscated and evasive malware. The lack of a complete understanding of malware behavior underscores the need for dynamic analysis, which involves executing malware in an isolated environment to analyse its behaviour at runtime to mitigate attacks. Many existing works in the area of Malware analysis fail to sufficiently explore dynamic malware features, which is the basis for building a high performance malware detection system. This paper provides a detailed and comprehensive overview of the state-of-the-art dynamic malware features relevant for robust malware detection or classifi-

*Corresponding author.
Email address: monday.onoja@fmph.uniba.sk (Monday Onoja)

cation. We systematically reviewed literature on dynamic Malware analysis and the features they utilize for machine learning-based malware analysis. The study also highlights the performance, contributions and limitations of various machine learning techniques proposed for dynamic malware analysis. It also identified and discussed different tools for dynamic malware analysis.This work not only serves as a compendium on the state-of-the-art in dynamic malware analysis but also provides a step-by-step framework for performing dynamic malware analysis. The improved taxonomy of malware analysis presented in the paper addressed contradictions in existing literature on malware analysis. In general, this paper provides trends and future directions in dynamic malware analysis to offer useful support for further research.

Keywords: Dynamic malware analysis, Malware features, Malware dataset, Behavioral malware analysis, Sanbox

---

.

1. Introduction

According to Cybersecurity Ventures, if measured as a country, cybercrime would be the third largest economy in the world after the United States and China, and it is predicted to cost the world approximately 10.5 trillion USD in 2025 [1]. The need to mitigate malware attacks cannot be overemphasized as malware authors consistently develop new and sophisticated techniques which continues to threaten cyber-space. Despite the substantive advancement of cybersecurity tools, malware attacks are still one of the most prevalent attacks presently [2]. The AV-TEST malware database is rapidly growing with over 1.55 billion registered malware and Potentially Unwanted Applications (PUA) as of January 2026, with over 1 billion of these samples as Windows malware [3]. Of the more than 104 million samples registered in 2025, about 101 million are Windows malware [3]. More than 3.2 million new malware samples were registered in January 2026. Approximately 450 thousand malware and PUA samples are registered daily [3].

It is therefore imperative to note that the Windows operating system is one of the major targets of malware attacks. The continuous rise in Windows malware attacks underscore the need to explore more security tools in

a bid to mitigate these attacks. Figure 1 shows the analysis of newly discovered malware and PUA samples per second, hour, day, month and year as registered by AV-TEST.

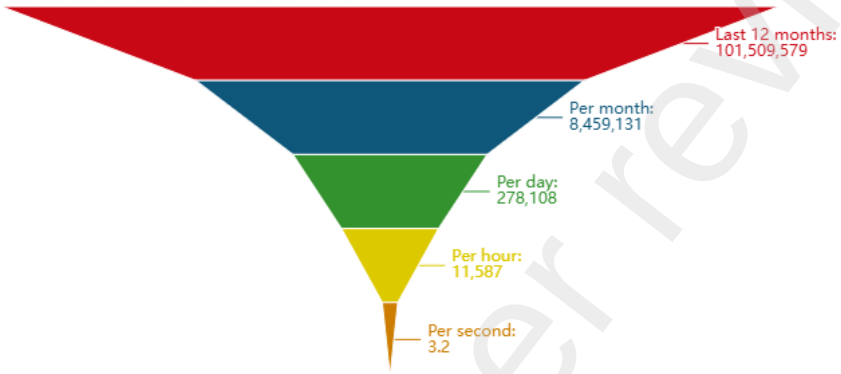

Figure 1: New malware and PUA per second [4]

While Figure 2 shows the total stock of malware and PUA collected by AV-TEST, subdivided according to their frequency of occurrence per operating system as captured by AV-TEST up to January, 2026 [4] . According to Statcounter Global Stats [5] as shown in Figure 3, the Windows operating system accounts for the majority of the global desktop OS market share (January 2021 to January 2026), around 70%, making it the dominant platform worldwide and incentivising attackers to continue developing malware that targets Windows-specific vulnerabilities due to the large base of active users.

Researchers are increasingly exploring the application of machine learning in developing defensive mechanisms. A major factor influencing the effectiveness of such mechanisms is the selection and utilization of appropriate features. Malware analysis utilizes diverse tools for program analysis to study malicious samples in order to gain a better insight into the behavior of malicious samples, including their evolving and evasive mechanisms [6]. Malware analysis methods may be static analysis, dynamic analysis, or hybrid analysis [7, 8]. Static analysis, also called static code analysis is achieved by examining and extracting data from a source code without executing it [9].

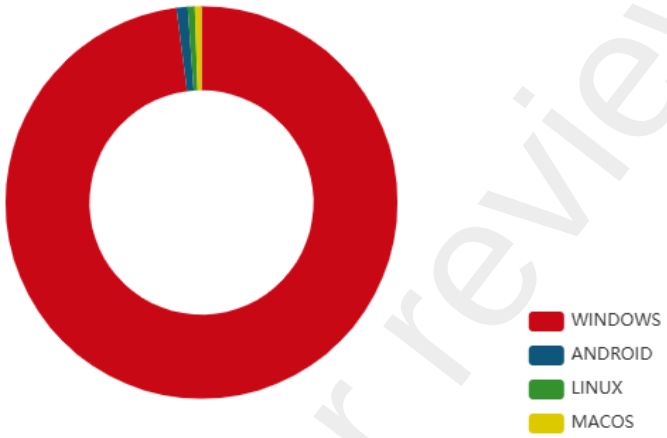

Figure 2: Malware Distribution by OS [4]

However, numerous anti–reverse-engineering techniques, such as code obfuscation, packing, and related protections, significantly hinder the effectiveness of static analysis. Dynamic analysis methods examine a program during execution, mostly in a virtual environment [9], enabling cybersecurity experts to capture runtime behaviors that cannot be captured statically. By running and examining malware behaviors, dynamic analysis addresses the shortfall of static analysis. While features obtained via static analysis have been predominantly utilized for malware detection, its limitations with respect to obfuscation [10], evasion tactics [11] and lack of understanding of malware behaviors [12] underscore the need for dynamic analysis in mitigating malware attacks. Extensive research has been conducted on dynamic malware analysis and the use of dynamic features for malware detection. However, several important gaps remain in the existing literature. First, there is a lack of comprehensive studies that systematically explore the different types of dynamic features, their combinations, and their comparative performance in malware detection. Second, inconsistencies, misconceptions, and contradictions persist in the conceptual understanding and taxonomy of malware and dynamic malware analysis, leading to confusion and lack of standardization in the field. Third, there is a notable absence of a well-curated and up-to-date repository of publicly available malware datasets, along with a detailed

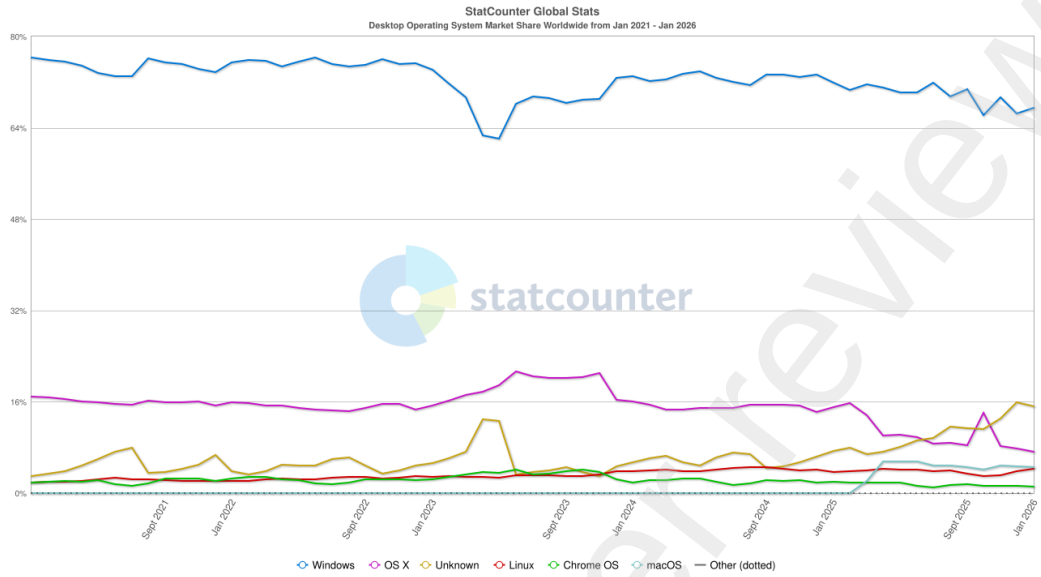

Figure 3: Desktop Operating System Market Share Worldwide [5]

guide and resource links to support researchers in conducting effective dynamic malware analysis. Addressing these gaps is crucial for advancing the state of malware detection and fostering reproducibility and collaboration within the cybersecurity research community.

The contributions of this study are as follows:

I. This study presents a comprehensive survey of state-of-the-art dynamic malware features, critically examining their significance, detection capabilities, and limitations across diverse machine learning-based classification and detection contexts.

II. To address ambiguities and inconsistencies in existing literature, this study proposes an improved taxonomy that systematically redefines and organizes key concepts in dynamic malware analysis, thereby promoting conceptual clarity and supporting standardisation in the field.

III. This paper introduces a structured and executable framework for dynamic malware analysis, outlining a clear and reproducible workflow from environment setup to behaviour capture and analysis, and serving as a practical guide for researchers and practitioners.

IV. The framework is complemented by a curated collection of tools and resources, including dynamic analysis environments, monitoring utilities, and a comprehensive, link-enabled catalogue of publicly available malware datasets mapped to each phase of the analysis process. This integrated resource supports reproducibility, benchmarking, and further research by improving dataset accessibility within the community.

The rest of the paper is structured as follows: Section 2 discusses malware and Malware analysis concepts. Section 3 reviews existing literature on malware analysis, while Section 4, presents malware features, machine learning techniques applied, respective use cases and datasets. Finally, Section 5 focuses on conclusions and recommendations of the study.

2. Background

2.1. Malware Overview

Malware, or malicious software, refers to any software that is intentionally designed to disrupt, damage, or gain unauthorized access to computer systems [13]. The landscape of malware is vast and ever-evolving, presenting a significant challenge for cybersecurity professionals. Traditional viruses and worms have given way to more sophisticated threats such as ransomware, spyware, and fileless malware, each with unique attack methodologies that exploit system vulnerabilities in different ways [14]. Understanding malware requires examining its classification based on its nature, behavior, and level of access. Some malware types, such as trojans and ransomware, deceive users into executing them, while others, like worms and botnets, spread autonomously across networks [15]. Additionally, malware can operate at different privilege levels, with user-level malware being easier to detect and remove, while kernel-level malware embeds itself deep within the system, making it much more elusive [16]. Figure 4 depicts common types of malware and their intentions or behavior.

2.2. Malware Analysis vs. Malware Detection

A common misconception in the literature is the interchangeable use of the terms malware detection and malware analysis. For instance, Sihwail et al. [17] treats "malware detection" techniques largely in terms of static versus dynamic methods, without clearly distinguishing them from broader "malware analysis" process. However, these are distinct, yet complementary

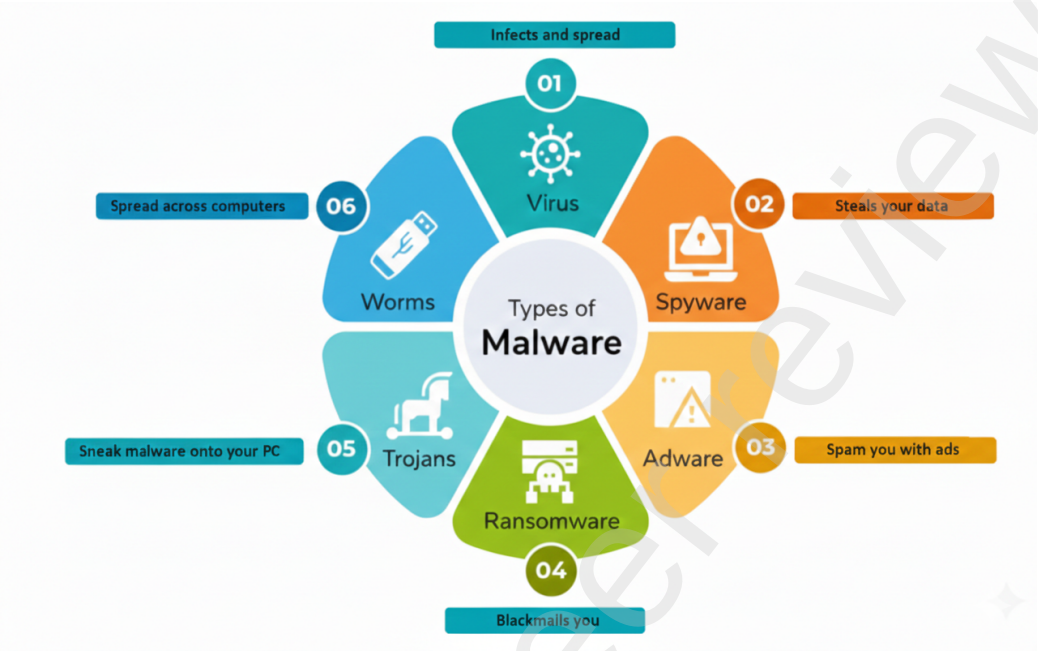

Figure 4: Types of malware and their intentions

processes. Malware detection focuses on identifying whether a file or program is malicious using methods such as signature-based, heuristic, and behavior-based detection [18]. Signature-based detection relies on predefined malware fingerprints, while heuristic detection analyzes code characteristics to identify new or modified threats. Behavior-based detection monitors the execution in real time to identify suspicious activities [8], this is the centrality of dynamic analysis, which is done at run-time. On the other hand, malware analysis delves deeper into understanding how a malware functions. It involves examining malware samples to determine their behavior, propagation methods, and impact on a system. This process is crucial in crafting robust countermeasures and enhancing detection capabilities. Without analysis, detection systems may struggle to keep up with rapidly evolving malware threats [19].

2.2.1. The Need for Malware Analysis

Malware analysis is a fundamental cybersecurity practice aimed at understanding the behavior, functionality, and impact of malicious software. The goal is to extract meaningful malware features that differentiate malicious files from benign ones [15]. Effective malware analysis enables security

experts to develop signature-based detection, behavioral models, and machine learning-driven classifiers to counteract modern cyber threats [20]. In general, malware analysis is categorized into *Static Analysis* which aims to examine the code, structure, and metadata of a malware sample without executing it [21]; *Dynamic Analysis* involves executing malware in an isolated environment (sandbox) to observe its behavior at runtime [22]; and *Hybrid Analysis* combines static and dynamic approaches to improve performance [23].

While static analysis is computationally efficient, it struggles against obfuscation and polymorphic malware, which alters its code to evade detection. Conversely, dynamic analysis provides real-time behavioral insights but is resource-intensive and vulnerable to sandbox evasion techniques [24].

2.3. Malware Features

Malware features are the distinct characteristics that security systems use to identify, classify, and predict malicious behavior. These features are broadly classified into static features and dynamic features [25].

2.3.1. Static Features: Analyzing Malware Without Execution

Static features are extracted without running the malware. This approach involves analyzing the malware's source code, binary structure, and metadata to infer potential threats [23]. The advantage of static analysis is that it is fast, computationally efficient, and safe, as there is no risk of activating the malware during examination [2]. However, it has notable limitations, particularly when dealing with obfuscated or polymorphic malware [26].

A widely used static feature is the opcode sequence, representing processor instructions. Malware often contains unique opcode patterns that distinguish it from legitimate software [24]. Additionally, embedded strings can reveal hardcoded URLs, API calls, or other functionality hints [27]. Static analysis also considers file metadata and headers, which may indicate suspicious structures [22].

2.3.2. Dynamic Features: Extracting Behavioral Indicators at Runtime

Dynamic analysis observes how malware interacts with the system at runtime. Dynamic analysis can be manual or automatic. Manual dynamic analysis (usually conducted with the aid of debugger) is an obsolete form of analysis, which is not applicable in all use cases due to its cost implications [26]. Automatic Analysis utilizes tools that examine and capture the behavior

of the analysed samples and its implication [26]. A critical feature used in malware detection is API call sequences, which reveal system interactions [2]. For example, ransomware typically makes calls to file encryption APIs, while keyloggers may interact with keyboard functions [23]. Registry modifications and process injections are also key indicators of malicious activity [26]. Many malware variants modify Windows registry keys for persistence, while others inject themselves into legitimate system processes to evade detection [24]. Additionally, network activity analysis can detect malicious communication with command-and-control servers [20].

Despite the increasing adoption of dynamic malware analysis, many existing studies fail to systematically explore dynamic features and their impact on machine learning-based malware detection [2]. Recent research suggests that combining static and dynamic features significantly improves detection accuracy [28]. However, feature selection methodologies remain inconsistent, and many malware datasets lack structured documentation on extracted dynamic features, leading to gaps in detection efficiency.

3. Previous Survey

Many previous studies on malware reviews are more general and do not tailor their efforts specifically to dynamic malware analysis or features. An in-depth review in this area is necessary, since dynamic analysis is the game-changer for capturing obfuscation and evasion techniques, which are often occluded in static analysis. A survey by Chakkaravarthy et al. [29] discussed malware analysis and mitigation techniques used by modern malware such as Advance Persistent Threats (APT). However, the authors did not consider extending the study to the Windows malware domain. A study by Jerlin & Jaya [30], only focused on the uses and role of Op-code frequency and n-gram for feature extraction in their attempt to explore Windows malware dynamic analysis. However, they completely ignored relevant dynamic malware features such as API call, which are useful for building effective malware detection system. In a bid to present a comprehensive survey on malware, the studies by Pachhala et al. [31], and Mira [25] presented a brief review on the available models for malware detection but failed to delve into the issues related to dynamic malware analysis and its features. Similarly, Al-Janubi and Altamini [32] presented a survey that informs the best feature extraction and classification approach for an overall best performance for malware detection. The authors listed API/System calls, N-grams, opcodes, strings and control

flow charts as the most extracted features for malware detection but could not present any insights into the contributions of features or provide details of the analysis method that produced them. In another study, Ismail et al. [33] highlighted static malware features to include Byte sequence strings, opcodes, and API calls but completely ignored features related to the dynamic analysis domain. Cletus et al. [34] presented the state of the art in malware research approaches without providing insight into dynamic malware features, which is the basis for building an effective malware detection system. The literature review by Kar & Bindal [35] tried to explore general malware features in Dynamic malware Analysis; however, the scope of the review was limited to only Windows XP and Dynamic Analysis using Cuckoo Sandbox. Besides the fact that the review was published in 2016, the state of the art in malware detection was not presented in the literature. Sihwail et al. [36] further compared malware detection tools based on the approaches and features they utilized. Although, the authors presented a comprehensive review of various malware features, there exists ambiguity in the taxonomy presented in their approach with the potential of possible contradiction. Ormier et al. [26] provided a very good insight into the methodologies, approaches and taxonomy of malware analysis, but they did not provide insight into useful dynamic features relevant for malware detection and their contribution based on machine learning models. In a more recent review, Gaber et al. (2024) [37] offered an in-dept systematic review on malware detection. The study provided a good insight into malware features and its relevance with respect to machine learning. While this systematic review reflects the state of the art, it fails to present any useful taxonomy for malware analysis which should present good understanding to researchers in the face of the prevailing contradictions in the malware domain. Also, Ucci et al. [38] presented a survey on malware analysis, but the taxonomy presented in the study indicates ambiguity since it fails to clearly map malware features to their respective analysis methods.

While these studies attempted to review existing works in dynamic malware analysis, none of them comprehensively delved into: a well conceptualized taxonomy devoid of ambiguity, a detailed discussion of malware features beyond surface features (such as Api calls, memory activities) to specific feature representations utilized in literature and their performance; provision of a comprehensive table of publicly available malware datasets and tools for dynamic analysis; a comprehensive list of frameworks (publicly available sandboxes), and practical steps for conducting dynamic analysis with map-

ping to the necessary resources for extracting desired dynamic features.

4. Methodology

The Systematic Literature Review (SLR) process adopted for this study is illustrated in Figure 6. The planning phase began with a comprehensive examination of existing SLRs and survey papers on dynamic malware analysis. These prior studies were critically assessed to extract key insights, uncover methodological limitations, and identify unresolved research gaps. This foundational analysis informed the development of the research questions guiding the present study.

To ensure a structured and comprehensive investigation, we conducted a systematic search for research publications from 2018 to 2026, focusing on studies related to dynamic malware analysis, feature extraction techniques, and classification mechanisms. The aim was to identify the most relevant feature sets, methodological approaches, and evaluation metrics used in contemporary malware detection research.

The selection process emphasised peer-reviewed journal articles, reputable conference proceedings, and high-impact publications across computer science, cybersecurity, and artificial intelligence.

Search Strategy

We conducted a structured literature search that returned a total of 4532 articles using the Scopus, Web of Science, and Google Scholar databases. The search queries were constructed to identify publications related to machine learning and deep learning techniques for malware detection based on dynamic or behavioral analysis. The search strings were as follows: Scopus: TITLE-ABS-KEY((malwar* AND detect*) AND (“dynamic analysis” OR “dynamic feature*” OR “behavioral analys*” OR “behavioural analys*” OR “runtime feature*” OR “execution trace*” OR “system call*” OR sandbox*) AND (review OR “systematic review” OR survey OR taxonomy OR framework OR “state of the art”) AND (“machine learning” OR “deep learning” OR classification OR model*)) AND (PUBYEAR $>$ 2017) AND (LIMIT-TO (LANGUAGE, "English")). Web of Science: TS=((malwar* AND detect*) AND ("dynamic analysis" OR "dynamic feature*" OR "behavioral analys*" OR "behavioural analys*" OR "runtime feature*" OR "execution trace*" OR "system call*" OR sandbox*) AND (review OR "systematic review" OR survey OR taxonomy OR framework OR "state of the art") AND

Table 1: Summary of Previous Survey

Paper	Main Focus	Dynamic Features Covered	Includes Taxonomy	Tools Listed	Public Datasets Provided	Dynamic Analysis Framework
[29]	Survey on malware analysis and mitigation techniques, with a focus on APTs and detection strategies.	Memory traces System calls API call counts Control flow graph RAM usage	Yes: taxonomy of malware mitigation and APT detection	- PEStudio - PEiD - Process Explorer - Cuckoo Sandbox - Noriben - Limon	No public datasets; samples sourced from internet repositories.	- Isolated virtual environments - NAT-separated networking - Sandboxing setup
[26]	Survey on dynamic malware analysis techniques, evasion, and ML robustness.	Function calls Volatile memory Network traffic CPU registers I/O and HPCs	Yes, taxonomies on behaviour, privilege level, and layout	- TTAnalyze - Anubis - Volatility - QEMU - PIN - DBI tools	No public datasets	- Bare-metal execution - VMs and Hypervisors - Emulation - Memory acquisition methods
[37]	Systematic review on malware detection using AI, covering malware sophistication, analysis techniques (static, dynamic), malware repositories, feature selection, and machine learning/deep learning.	- API calls - Opcode sequence - Flow control - Windows registry interactions - CPU registers - File operations - Network activity logs/traffic - HPCs - Memory dumps - System calls	No new taxonomy; discusses malware types (trojans, ransomware, etc.), evasion techniques (92 in 16 categories), and anti-instrumentation methods.	- Intel Pin - Cuckoo Sandbox - SecBox - Wireshark - perf - OllyDbg - VirusTotal	- EMBER (1.1M static PE) - BODMAS (134k PE, categorized) - SOREL-20M (20M samples) - VirusShare (55M samples) - Malware Bazaar (700k malware w/ metadata)	- DBI environments - Sandboxes (Cuckoo, SecBox) - VMs for malware execution - JSON reports from Cuckoo - Notes DBI's high fidelity but computational cost
[30]	Survey on dynamic malware analysis for Windows platforms, identifying general malware features using Cuckoo Sandbox. Suggests opcode frequency and n-gram analysis for feature extraction.	- System interaction - System effects - File system changes - Registry changes - Mutex creation - Packers and anti-debugging - Function parameters - Information flow tracking - Win32 API calls - Network traffic (PCAP) - Memory dumps	Includes classification of malware detection techniques, distinguishing between static and dynamic analysis as well as a comparison of runtime-behaviour with defined-point comparison.	- SysAnalyzer - Process Explorer - FileMon - Regmon - Regshot - TCPVIEW - TDIMon - Ethereal - Cuckoo Sandbox - Anubis - Andrubis - GFI Sandbox	No public datasets; references data from Honeynets for feature extraction.	- Simulated environment (VM) - Cuckoo Sandbox on Windows XP SP3 - State-diff analysis - Real-time monitoring tools used during execution
[31]	Comprehensive survey on machine learning methods for malware detection, focusing on deep learning, challenges, and future research directions.	System interaction Application programs Hooking detection Network services Rootkits link Hidden objects Injection code Runtime trace reports Memory analysis Network traffic Dynamic instructions System call outputs	Classifies malware types such as viruses and worms based on function.	- PEiD, ssdeep - pafish, Yara - strings, IDA Pro - OllyDbg, OllyDump - Volatility - PIN tools - Valgrind - Cuckoo Sandbox - PinFWSandbox - Python in Apache Spark - WEKA	0.7M files (0.18M clean, 0.61M malware) from VXHeaven, Nothing, VirusShare; VirusTotal API also referenced.	- Automated dynamic analysis via Cuckoo Sandbox - PinFWSandbox for dynamic and log data collection
[?]	Review of malware detection strategies based on API call sequences, focusing on characterising malware behavior, detecting variants, and discussing accuracy/runtime limitations.	Malicious API calls API call sequences System files Processes and threads Devices Windows registry Network services via API calls Common substrings in dynamic traces	No, lacks formal taxonomy.	- IDA Pro (static API extraction) - API Monitor (dynamic API extraction) - CUDA (for performance acceleration)	CSMINING (API call sequences); dataset of malware and clean PE files The referenced study used 4,684 malware samples and API sequences from 1,821 files.	- Execution in simulated environments (e.g., VMs) - Comparing system states pre/post execution - Observing runtime behavior with tools like API Monitor - Multiple executions for full path coverage
[32]	Comparative analysis of machine learning techniques for malware detection, focusing on feature extraction and classification methods, with static, dynamic, and hybrid approaches for Windows and Android systems.	Program's behavior API/System calls Battery and CPU consumption Number of messages and running processes SMS, phone calls, network I/O (Android via DroidBox)	Yes, malware types (Viruses, Trojans), detection methods (Signature-based, Behavior-based), and analysis techniques (Static, Dynamic, Hybrid)	- DroidBox (Android sandbox) - Cuckoo Sandbox (Windows sandbox) - Androguard (Android decoder) - 7-zip (APK uncompression) - WEKA (ML tool)	Yes, datasets created by authors (Android malware), benign apps from Google Play, malicious from Contagio, Genome Project, OpenMalware, Softonic	- Simulated environments with VMs - DroidBox virtual environment for Android - Cuckoo Sandbox for Windows - Andromaly framework (Android behavioural detection)
[34]	Scoping review evaluating current malware trends, attack and defense strategies, including obfuscation, and research gaps in ML-based malware detection.	Runtime behavior CPU, memory, and network usage API arguments Network traffic Processor consumption Battery usage System calls	Yes, malware detection schemes (Signature-based, Behavior-based, Heuristic-based, Anomaly-based, Statistics-based); obfuscation techniques (anti-static, advanced)	General terms like Anti-virus and scanners; no specific dynamic tools listed but surveys tools used in literature	No public datasets provided; discusses need for empirical testing with real-time obfuscation methods	General dynamic analysis as observing program execution; time-consuming and environment-dependent; notes hybrid static-dynamic approaches; future directions: data augmentation, CNNs, GANs

("machine learning" OR "deep learning" OR classification OR model*)) AND PY=(2018-2026) AND LA=(English). Google Scholar: ("malware detection" OR malware detect*) AND ("dynamic analysis" OR "behavioral analysis" OR "system call" OR sandbox OR "execution trace" OR "runtime feature") AND ("machine learning" OR "deep learning" OR classification OR model*) AND (review OR survey OR "systematic review" OR "state of the art" OR taxonomy OR framework).

The Google Scholar results were filtered using the year range *2018–2026* and language *English* to align with the other databases.

Inclusion and Exclusion Criteria

The inclusion criteria for this study required that selected papers employ a quantitative research design using experimental methods to evaluate artificial intelligence techniques with a specific focus on malware analysis and detection, and that they address dynamic or behavioral features relevant to malware detection such as system calls, execution traces, sandbox behavior, or runtime characteristics, and be published in or after 2018 in peer-reviewed outlets. To ensure uniformity and relevance, only studies written in the English language were considered, while documents published before 2018, those that did not pertain to malware detection or analysis, and non-peer-reviewed material such as editorials, abstracts without full papers, magazine articles, white papers, or reports were excluded. Following the application of these criteria, a total of 107 relevant articles were identified and selected for final analysis. This structured approach to inclusion and exclusion is consistent with established practices in systematic literature review methodology, which emphasize the importance of defining criteria that clearly delineate eligible studies to focus the review on relevant and methodologically appropriate research. The PRISMA flow diagram in Figure 5 stows the study identification, screening, eligibility assessment, and inclusion for this systematic review.

5. Malware Detection System

This section delves into the critical dynamic features leveraged in malware analysis, with a focus on their nature, significance, extraction methodologies, and performance.

Figure 7 presents a structured taxonomy mapping the relationships between malware defense mechanisms, malware sample acquisition, malware analysis, and malware detection. The blue arrows in the first layer highlight

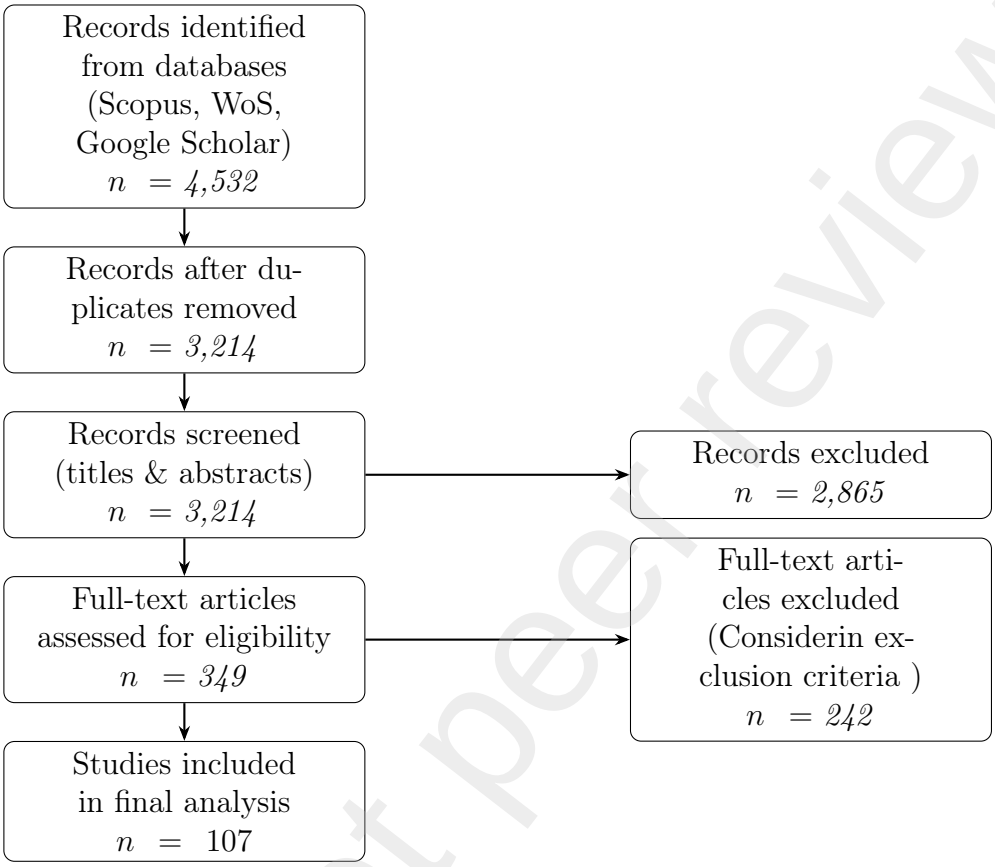

Figure 5: PRISMA flow diagram

the malware detection cycle, which begins with either a pre-processed malware dataset (ready for direct ingestion) or live malware samples that must first be analyzed to extract discriminative features. Analysis is accomplished through static, dynamic, or hybrid techniques, each aligned with supporting tools and yielding characteristic feature sets; static analysis (e.g. file hashing, opcode sequences, API call), dynamic analysis (e.g. API-call logs, system modifications captured in sandbox, DLL), or hybrid methods that combine both to enhance detection robustness. These features are fed into the malware detection layer, which encompasses signature-based, heuristic/anomaly-based, and behavior-based detection approaches that operate on static, dynamic, or hybrid data. While the full pipeline is depicted for conceptual completeness, the focus of this paper lies squarely on the analysis layer, ex-

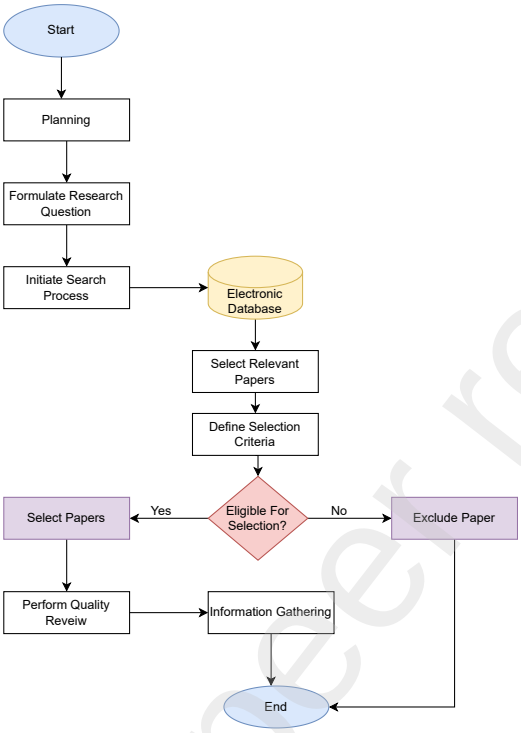

Figure 6: Systematic Review

ploring the tools, techniques, and feature extraction methods that precede and enable detection and not the detection approach/algorithms themselves.

5.1. Malware Sample

Malware samples are fundamental to the analysis and detection of malicious software. They serve as the raw material upon which detection models are built and refined. The acquisition process can be categorized into two main types: datasets of already extracted samples and live malware samples. *Pre-Processed Malware Datasets:* Publicly available datasets of already extracted malware samples provide researchers with a curated collection of samples that have undergone initial analysis and feature extraction. These datasets allow for immediate application in building detection models, where the focus shifts primarily to feature engineering and algorithm optimization. Table 2 presents a comprehensive list of publicly available malware datasets of pre-processed samples. This approach streamlines the process, leveraging existing data to expedite research and development efforts.

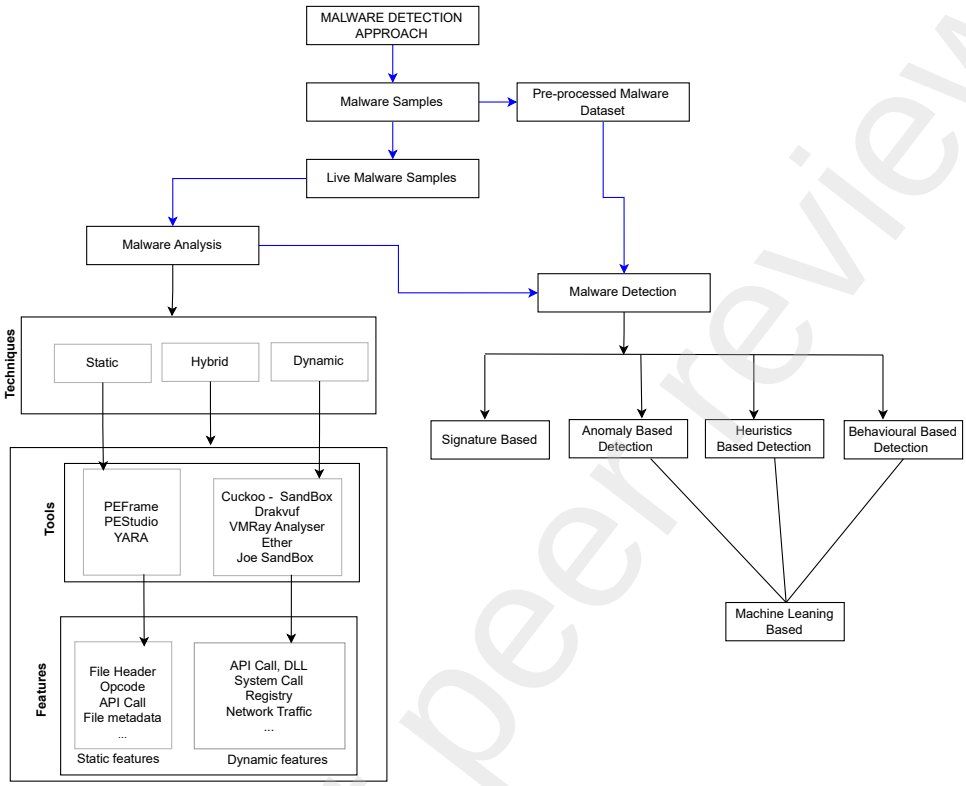

Figure 7: Malware detection approach

Live Malware Samples: Live malware samples, often sourced from malware zoos or through targeted collection efforts, offer a different perspective. These samples are raw, unprocessed instances of malware that require comprehensive analysis, both statically and dynamically, to extract relevant features for detection purposes. Unlike datasets of pre-extracted samples, working with live malware enables researchers to explore novel features and adapt detection strategies to emerging threats. Table 3 provides a comprehensive list of publicly available zoos for live malware samples. While the flexibility of live samples encourages innovation in feature extraction, it also introduces additional complexities and costs associated with thorough analysis. Researchers must balance the benefits of feature customization with the practical challenges of processing and securing live malware samples.

In summary, the choice between utilizing datasets of pre-extracted sam-

ples and working with live malware samples represents a strategic decision in malware analysis and detection. Each approach offers unique advantages and challenges, shaping the development of effective detection systems in response to evolving cybersecurity threats.

Table 2: Publicly Available Malware Datasets

Dataset Name	**Type**	**Sample Description**	**Source**
Virus-MNIST	Grayscale malware images	51,880 samples across 10 classes	https://0xh3xa.github.io/awesome-malware-benign-datasets/
Malimg	Malware images	9,458 images from 25 malware families	https://0xh3xa.github.io/awesome-malware-benign-datasets/
Stamina	Binary-to-image conversion	782,224 binary sequences converted to images	https://0xh3xa.github.io/awesome-malware-benign-datasets/
IoT DDoS	IoT malware binaries	365 samples with 3 types of IoT DDoS attacks	https://0xh3xa.github.io/awesome-malware-benign-datasets/
2018 EMBER	Feature vectors (CSV)	900K training + 200K test PE features	https://github.com/elastic/ember
2024 EMBER	Feature vectors (CSV)	2.6M training + 600K test	https://github.com/FutureComputing4AI/EMBER2024
Microsoft Malware Prediction	PE malware feature dataset	Structured features in CSV format	https://0xh3xa.github.io/awesome-malware-benign-datasets/
BIG 2015 (Microsoft)	PE binaries + disassembly	20K malware samples with bytecode and labels	https://arxiv.org/abs/1802.10135
CICIDS 2017 / 2018 / 2019	Network traffic logs	Benign + malicious labeled network flows	https://www.unb.ca/cic/datasets/index.html
MalNet-Image	Malware images	1.2M images across 696 malware families	https://doi.org/10.1145/3511808.3557533
SOREL-20M	Features + disarmed binaries	20M feature vectors + 10M disarmed PE binaries	https://arxiv.org/abs/2012.07634
BODMAS	PE binaries + metadata	57,293 malware + 77,142 benign PE files	https://gangw.web.illinois.edu/data.html
MalMem2021	Memory dumps	Memory snapshots of benign and malicious processes	https://0xh3xa.github.io/awesome-malware-benign-datasets/
AndroZoo	Android APK malware	24M APKs with antivirus detection labels	https://androzoo.uni.lu/
Benign-Net	Binaries of PE benign samples	contains a huge amount of normal .NET exe files	https://github.com/bormaa/Benign-NET
Malware Genome Project	Android malware samples	1,260 samples across 49 malware families	https://link.springer.com/article/10.1007/s44248-023-00003-x
Malicia Dataset	Windows PE binaries	11,688 malware binaries from drive-by downloads	https://link.springer.com/article/10.1007/s10207-014-0248-7

5.2. Malware Analysis Techniques

Malware analysis is essential for understanding, detecting, and mitigating malicious software threats. The main techniques includes static, dynamic,

Table 3: Publicly Available Live Malware Samples

Category	Dataset	Features	Source
Windows Malware	MalwareBazaar	Large repository of Windows malware samples, updated daily	https://bazaar.abuse.ch/
	VX-Underground	35M+ Windows malware samples and APT toolsets	https://vx-underground.org/
	VirusShare	Over 40M malware samples, downloadable in archive packs	https://virusshare.com/
	Malshare	Free repository of Windows malware samples with API access	https://malshare.com/
	Das Malwerk	Continuously updated malware feed for Windows samples	https://dasmalwerk.eu/
	Hybrid Analysis	Downloadable PE binaries + sandbox behavioral reports	https://www.hybrid-analysis.com/
	InQuest Labs	Repository of malicious PE files, scripts, and documents	https://labs.inquest.net/
	theZoo	Malware binaries + source code for reverse engineering	https://github.com/ytisf/theZoo
Android Malware	AndroZoo	24M+ Android APKs with antivirus detection metadata	https://androzoo.uni.lu/
	Malware Genome Project	1,260 Android malware samples across 49 families	https://link.springer.com/article/10.1007/s44248-023-00003-x
	theZoo Dataset	Android malware binaries + source code	https://github.com/ytisf/theZoo
Network-based Malware	Malware Traffic Analysis	PCAPs, malware samples, and incident reports	https://www.malware-traffic-analysis.net/
	CICIDS 2017/2018/2019	Benign + malicious network traffic datasets with labels	https://www.unb.ca/cic/datasets/index.html
	URLHaus	Continuously updated malicious URL database + payloads	https://urlhaus.abuse.ch/
General-purpose	VirusTotal Intelligence	Billions of malware samples + AV detections and behavioral analysis (API required)	https://www.virustotal.com/
	Any.Run Malware Trends	Interactive sandbox with downloadable samples and IoCs	https://any.run/malware-trends/
	MISP APT Samples	Curated APT malware samples + indicators of compromise	https://github.com/MISP/malware-samples
	Contagio Malware Dump	Malware samples, phishing kits, and exploit payloads	http://contagiodump.blogspot.com/
	ZooLab	Live samples curated by malware researchers	https://zoolab.io/

and hybrid analysis, each offering unique strengths and limitations. Recent research has focused on comparing these methods and developing integrated approaches to improve detection accuracy and resilience against evolving malware.

Static Analysis: Examines malware without execution, analyzing code, metadata, and structure. It is fast and effective for known threats but struggles with obfuscated or novel malware [17, 39, 40, 41].

Dynamic Analysis: Observes malware behavior during execution in a controlled environment, capturing runtime activities like API calls and network traffic. It is robust against obfuscation but resource-intensive and can be evaded by sophisticated malware [42, 43, 26, 44, 40].

Hybrid Analysis: Combines static and dynamic features to leverage the strengths of both, aiming for higher detection accuracy and broader threat coverage [43, 39, 45, 46, 47, 48].

Fully dynamic analysis often achieves the highest detection rates but is costly for large-scale deployment [42, 26]. Since the focus of this study is to highlight and discuss the useful dynamic malware features and their performance in the literature, subsequent sections of this work will be focus on dynamic malware analysis and features.

5.3. Dynamic Analysis

As discussed earlier, dynamic analysis is a fundamental methodology in malware detection that involves executing a suspicious sample in a controlled, isolated environment [49, 43]. This process allows researchers to observe the malware's runtime activities and collect concrete records of its behavior, known as dynamic features, without the risks of a live system [50].

It is important to clarify the relationship between dynamic analysis and behavioral analysis, as they are two sides of the same coin. Dynamic analysis refers to the technical process of executing the malware and gathering raw runtime data (the "what" e.g., API calls, file changes). Behavioral analysis is the next step, focused on interpreting this data to understand the malware's intent and patterns (the "so what"). This deep understanding, in turn, fuels behavioral detection, which uses these identified patterns to create automated systems for classifying new, unknown samples.

Thus, dynamic analysis is the data collection phase, and behavioral analysis is the characterization phase that gives the data meaning. In the following subsections, we will delve into the various categories of dynamic (behavioral) features prevalent in the literature.

5.4. Dynamic Features

This section delves into the critical dynamic features leveraged in malware analysis, with a focus on their nature, significance, extraction methodologies, and performance. We begin with a structured taxonomy (Figure 7) that outlines the relationship between malware defense mechanisms, feature extraction processes, and types of malware analysis. Following this, we examine key dynamic features such as API calls, DLL interactions, registry modifications, and behavioral patterns. For each feature, we discuss its role in malware detection, the tools and techniques used for extraction, and its performance in malware defense based on literatures.

Clarifying the Relationship Between API Calls, Function Calls, and System Calls

To ensure conceptual clarity in the discussion of dynamic features, it is essential to distinguish and highlight the relationships between API calls, function calls, and system calls, as these terms are often used interchangeably or inconsistently in the literature. A *function call* refers generally to the invocation of any routine within a program, including both user-defined and library-provided functions. An *API call* is a specific type of function call that invokes predefined interfaces exposed by system or external libraries, such as the Windows API. These often wrap one or more *system calls*, which are low-level requests made directly to the operating system kernel to perform tasks like file access or process management.

In this hierarchy, API calls abstract system calls, and function calls encompass both. For example, a call to a user-defined function such as processData() is considered a function call, while `CreateFile()` is an API call that internally triggers the system call `NtCreateFile`. While system calls offer a more granular and reliable view of actual system interactions, API and function calls provide higher-level behavioral context. In dynamic malware analysis, leveraging all three can yield complementary insights into program behavior.

5.4.1. API Calls

API calls are fundamental indicators of a program's behavior, offering critical insights into how software interacts with the operating system and other resources. Whether obtained through static analysis, by examining code structures and identifying invoked functions, or through dynamic analysis, by monitoring runtime behaviors, the API call patterns help reveal a

program's intentions. This makes API call analysis a cornerstone in malware detection, enabling the identification of malicious activities across various analysis techniques.

An Application Programming Interface (API) call is a request made by a program to the operating system or another software component to perform a specific function, such as reading a file, allocating memory, or sending data over a network. In the context of malware analysis, observing API calls during execution helps in understanding the actions a program takes, revealing its true intent [51]. API calls are the "DNA" of program behavior in Windows (or other OS environments). Malware often abuses APIs to achieve malicious goals (e.g., file encryption, process injection, registry manipulation). API calls are a valuable feature in malware defense because they provide direct insight into a program's behavior during execution.

Although some studies [42, 52] exclusively associate the API call with dynamic analysis, it is important to note that the API call features can be accessed via both static and dynamic analysis. In static analysis, API calls are extracted by examining the program's code or binary without execution. This typically involves parsing the Import Address Table (IAT), analyzing function call instructions in disassembled or decompiled code, and detecting API usage patterns through string references or code signatures. These features help reveal the program's intended interactions with the operating system and are commonly used in malware detection and behavioral classification [15, 53, 54]. However, static analysis has limitations, especially when dealing with obfuscated code or packed binaries. Malware authors often use these techniques to hide malicious code, making it challenging for static analysis to accurately identify API calls and their intended behavior [55, 56].

API Calls via dynamic analysis

Dynamic analysis allows for the monitoring of actual API calls made during execution, providing insights into the program's runtime behavior. Dynamic analysis is particularly useful for detecting behaviors that are not evident in static code, such as API calls generated at runtime or those that depend on specific execution paths [57, 58]. While static analysis can identify the presence of API calls within code, it is inherently limited to potential behaviors, as it does not account for runtime context [59]. In contrast, dynamic analysis captures the actual execution of code, revealing several features that static methods cannot reliably uncover. These include the precise sequence

and timing of API calls, the actual system resources accessed, and the concrete files created, modified, or deleted during execution. Static analysis may list file-handling functions, for example, but only dynamic analysis can confirm whether and how these functions were invoked. Furthermore, dynamic analysis can detect runtime-resolved APIs, environment-dependent behavior, and anti-analysis techniques in action, offering a more accurate representation of a program's real-world behavior. efforts are ongoing to be able to access some of these dynamic features statically. For instance, tools like Apícula have been developed to statically detect API calls in generic streams of bytes, such as memory dumps or object code files [60].

API call sequences and patterns are critical behavioral indicators in distinguishing malicious software from benign programs. While both categories may invoke similar APIs, differences often lie in the order, frequency, and contextual relationships among these calls. These behavioral signatures can be effectively leveraged to differentiate benign from malicious processes [**?** 61, 62]. API call frequency, in particular, offers valuable insights into a program's intent and interaction with system resources. Numerous studies have demonstrated that incorporating Windows API features significantly enhances malware detection model performance [63, 64, 65, 2, 66, 67, 68]. For example, Syeda and Asghar [63] employed a dual-layered feature selection approach using the Chi-square (Chi2) and Gini importance. Their method effectively identified key API calls such as `NtAllocateVirtualMemory`, `NtProtectVirtualMemory` and `LdrGetProcedureAddress` for malware discrimination, leading to improved precision and detection accuracy across several machine learning algorithms. Zhaoqi et al. [69] proposed a low-cost feature extraction technique utilising the Cuckoo Sandbox to execute Portable Executable (PE) files in a virtualized environment. API hooking was employed to monitor 312 API calls, grouped into 17 functional categories. To reduce redundancy, API names, categories, and arguments were hashed using MD5. These transformed features were input into a Gated CNN-BiLSTM pipeline, enabling sequential pattern learning and outperforming baseline models. By extracting semantic features from API names and arguments using lightweight feature techniques, resulting in an effective malware detection and classification framework, Li et al. [2] analyzed more than 38,000 malware samples using the Cuckoo Sandbox and highlighted the critical role of memory-management APIs, which are frequently targeted by malicious programs. The authors also extracted semantic information from both API names (e.g., `NtCreateFile`) and their heteroge-

neous arguments (e.g., `status info`), an aspect often overlooked in related work. The resulting API call sequences were transformed into API call graphs, and a hybrid GNN architecture combining GINE and GAT was proposed to jointly capture semantic content and structural relationships within these graphs. Memory-related APIs such as `NtProtectVirtualMemory` and `NtAllocateVirtualMemory` were identified as high-impact features in Syeda and Asghar's study [70], where filtered API call features were selected using embedded methods. Recognizing evasive tactics like API padding, Prachi et al. [66] first extracted frequent API patterns and then applied Global Local Best Particle Swarm Optimization (GLBPSO) for feature selection. They further enriched the model with a Genetic Algorithm (GA), achieving detection accuracy up to 100% in certain cases. Their tool, MalAnalyser, was particularly effective against ransomware. D'Angelo et al. [71] introduced a novel classification framework based on association rules and frequent subsequences of API calls. Their approach modeled the transition probabilities and timelines of API sequences, outperforming established methods such as Markov chains and sequence alignment in distinguishing malware families. Several studies also employed natural language processing (NLP) techniques to represent API sequences. Maniriho et al. [72] propose a hybrid automatic feature-extraction architecture that integrates a Convolutional Neural Network (CNN) with a Bidirectional Gated Recurrent Unit (BiGRU). This combination enables the model to capture both local spatial patterns and long-range temporal dependencies in raw, extended sequences of API calls, thereby effectively distinguishing malicious behaviors from benign activities. In addition, the authors provide a benchmark dynamic-analysis dataset derived from API-call traces, supporting further research in behavior-based malware detection. Amer and Zelinka [73] proposed a dynamic prediction framework that combines word embeddings and Markov chains. By clustering API calls and modeling their sequences, they achieved early-stage malware prediction with high accuracy. Tohru et al. [74] focused on two specific indicators, frequent API calls and memory allocation statistics using SVMs to reach 82.7% accuracy and 84.2% F-measure, showing the discriminative power of these features. Zhijie et al. [65] translated API call sequences into numerical vectors using embeddings and clustering, then trained a Text-CNN binary classifier to distinguish between malicious and benign software. This dynamic technique demonstrated strong performance, especially for evolving threats. Similarly, Zhang et al. [75] propose a deep-learning-based malware detection method that leverages the behavioral richness of API call sequences

by converting them into grayscale images and performing classification jointly with sequence features. Their approach uses a variety of neural networks to extract both semantic and graphical characteristics from the API sequences, enabling the model to capture nuanced behavioral patterns and achieve high detection accuracy.

Semantic Information of API Call

Li et al. [76] argue that relying solely on basic API sequence features such as API names, call frequencies, and usage statistics is insufficient for robust malware detection, particularly given the increasingly sophisticated and evasive techniques adopted by modern malware. To address this shortcoming, the authors proposed a deep learning-based detection framework that incorporates the intrinsic semantic characteristics of API sequences. Specifically, the model captures the software execution context, the semantic information embedded in the API sequences, and the interdependencies among API calls. These features offer more informative and discriminative signals for dynamic malware detection. The framework was evaluated on a large dataset comprising 20,887 malware samples and 22,120 benign samples obtained from sources such as VirusShare, Softonic, SourceForge, and PortableApps, with all samples validated using VirusTotal to ensure dataset integrity. Building on the importance of semantic features, Mal-ASSF [77] emphasizes not only the sequential structure of API calls but also the semantic operations and resource types associated with them. Unlike earlier approaches that treated API calls as isolated numerical tokens, Mal-ASSF integrates both sequential patterns and semantic content, enabling more accurate classification. This hybrid approach significantly outperforms existing methods in identifying diverse malware families, highlighting the value of semantically enriched representations. To further investigate the semantic relationships and latent structures in API call sequences, Voronin and Morozov [67] treat API call sequences as text. Their approach aims to establish semantic relationships between API calls, identify stable n-grams (collocations) indicative of malicious activity, and perform thematic modeling to cluster and classify different malware types. The authors employ word embeddings to group semantically similar API calls and apply Latent Dirichlet Allocation (LDA) for thematic modeling. Expanding this idea into graph-based representation, the DMal-Net framework [2] performs API feature engineering to extract semantic information from both API names and their arguments. It then constructs an API call graph, where nodes represent API calls and edges encode their rela-

tional semantics. A Graph Neural Network (GNN) is used to analyze these structural graphs, capturing complex interactions and dependencies among API calls. The proposed method exhibited superior performance in malware detection and classification tasks using a large-scale dataset comprising 18,423 positive samples spanning eight malware types and 20,149 negative samples collected from four distinct sources. These results validate the effectiveness of integrating semantic API features with graph-based deep learning for enhanced malware analysis.

Integrated API Call

Namita et al. [78] introduced an Integrated API Call Feature Set for Windows malware detection by combining API usage, frequency, and sequence extracted via dynamic analysis. To improve detection performance, they applied TF-IDF weighting, and tested the enriched features using various machine learning algorithms. Among these, Support Vector Machine and Logistic Regression achieved the highest accuracy. Addressing the limitations of conventional API-only approaches, the CTIMD model [79] incorporated Cyber Threat Intelligence (CTI) to extract Indicators of Compromise (IOCs) and label API parameters based on security sensitivity. These enriched API features, when input into deep learning models, significantly improved the detection of unknown malware compared to models relying solely on API names or unlabeled parameters. Further broadening the scope, the study Going Beyond API Calls in Dynamic Malware Analysis [80] used a custom dataset from the Cuckoo Sandbox to compare Random Forest models trained on full behavioral data versus API-only features. Results showed that incorporating a wider range of dynamic features led to markedly better detection accuracy, especially in cases where malware evaded API-based triggers. To overcome limitations in current malware analysis, Yang et al. [81] propose a method based on dynamic malware analysis using API calls and Supervised Contrastive Learning (SCL). Their framework encodes both semantic and statistical features for each sample, dynamically compares samples across different categories, learns inter-sample differences, and performs classification. In particular, the authors combine API names with their corresponding parameters to construct API sentences enriched with semantic information, and propose a hybrid feature encoder to jointly capture the semantic and statistical characteristics of API parameters.

5.4.2. System Calls

System calls are low-level interfaces through which user-space applications request services from the operating system's kernel [82]. Unlike higher-level API calls, which may wrap multiple operations or abstract away kernel interactions, system calls offer a more granular and often more tamper-resistant view of program execution. In malware detection, system call sequences provide rich behavioral signatures that are difficult to obfuscate, making them highly valuable for dynamic analysis. As a result, they have become a core feature in numerous behavioral-based malware classification models, especially those leveraging machine learning and deep learning techniques [83, 84].

A *system call* is a programmatic way for a user-space application to request services from the operating system's kernel, such as file I/O, memory allocation, or process control. In Windows, system calls often appear as `Nt*` or `Zw*` functions (e.g., `NtCreateFile`, `NtOpenProcess`), while on Linux, they include calls like `open`, `execve`, or `read`. These calls represent the lowest level of interaction a program can have with the OS without breaching privilege boundaries [85].

In the context of malware, analyzing system calls provides insight into core behaviors such as process creation, file manipulation, and memory access. Because they bypass user-mode abstractions, system calls are less susceptible to deception by packed or obfuscated code. Recent studies demonstrate that deep learning models trained on system call sequences achieve high accuracy in classifying malware families [86, 87]. Moreover, the structure, order, and frequency of these calls serve as behavioral fingerprints unique to malware types or variants [88].

System Calls via Dynamic Analysis

System calls are primarily observed via dynamic analysis, which monitors program execution in a controlled environment such as a sandbox or virtual machine. Tools such as `strace` [89] and `Linux AuditD` perform native system call tracing, capturing syscall invocations, arguments, return values, and calling processes, whereas `Sysmon` [90] and `Cuckoo Sandbox` [91] primarily monitor higher-level process and API activity from which system call behavior may be inferred. By capturing these runtime traces, analysts can construct system call graphs, n-gram sequences or other engineered features to feed into machine learning classifiers [92, 93]. Unlike API calls, system calls are harder to hook or replace, offering more trustworthy insight

into malware behavior [94, 95]. For instance, while a malware sample might obfuscate its API usage, its system-level operations (e.g., opening a file or allocating memory) still produce detectable kernel interactions. Advanced techniques such as system call hooking detection and behavioral profiling using system call frequency distributions help identify stealthy or polymorphic malware [96]. In a similar behavioral analysis study, Badis et al. [97] used a voting ensemble to analyze system call patterns, recommending the integration of such behavior-based methods into Endpoint Detection and Response (EDR) systems to combat sophisticated malware.

5.4.3. Function Calls

Function calls represent the invocation of specific routines or subroutines within a program, encompassing both user-defined (created by a programmer/developer) and library-provided functions. This invocation is achieved by an instruction that invokes a specific function within a program's codebase. Functions are blocks of code designed to perform specific tasks. They are called (invoked) from various points in a program to execute their code and return control to the caller when finished [98, 99]. In the context of malware analysis, monitoring function calls can provide insights into the program's control flow and behavior. While function calls can include API calls [100], they also encompass internal functions that may not interact directly with the operating system, offering a more granular view of program execution. These functions can be part of the standard libraries, third-party libraries, or user-defined within the application [99].

Function calls may reference functions from standard libraries, third-party libraries, or user-defined components within an application. Hence, monitoring function calls allows analysts to trace the execution path of a program, identify the sequence of operations performed, and detect any anomalous behaviors indicative of malware [101, 102]. For instance, certain malware may employ specific sequences of function calls to obfuscate their true intent or to perform malicious activities such as data exfiltration or privilege escalation. Recent studies [103, 104, 105, 106] have highlighted the effectiveness of analyzing function call sequences and graph for malware detection.

Function Calls via Dynamic Analysis

Dynamic analysis involves executing a program in a controlled environment to observe its behavior in real-time. By integrating monitoring mechanisms into the runtime environment, analysts can monitor function calls

as they occur, capturing information such as the function name, parameters passed, return values, and the sequence of calls [49, 107]. Tools like `DynamoRIO` [108] and `Intel Pin` [109] facilitate such monitoring by providing hooks into the program's execution flow.

Analyzing function call sequences dynamically allows for the detection of behaviors that may not be evident through static analysis. For instance, malware may employ code injection techniques or dynamically resolve function addresses at runtime to evade static detection methods (as shown in Listing 1). By observing the actual function calls made during execution, dynamic analysis can uncover such evasive behaviors.

```
#include <windows.h>

int main() {
    // Dynamically resolve the address of MessageBoxA
    HMODULE hUser32 = LoadLibrary("user32.dll");
    if (hUser32 != NULL) {
        FARPROC pMessageBoxA = GetProcAddress(hUser32, "
            MessageBoxA");
        if (pMessageBoxA != NULL) {
            // Call MessageBoxA indirectly
            ((int (*)(HWND, LPCSTR, LPCSTR, UINT))
                pMessageBoxA)(
                NULL, "Malicious Behavior", "Warning",
                    MB_OK);
        }
    }
    return 0;
}
```

Listing 1: Illustration of a case where a function (MessageBoxA) is resolved dynamically at runtime using LoadLibrary and GetProcAddress to obscure malware behavior from static analysis tools, which may fail to detect the actual function call.

Advanced detection models leverage the temporal and semantic relationships between function calls. For example, SeGDroid [110] leverages the semantic knowledge from sensitive function call graphs (FCG) to detect Android malware. They [110] defined sensitive function call graph as a pruned version of a standard FCG that retains only the nodes containing security-related APIs (e.g., accessing contacts or sending SMS) and all other nodes along the function call paths that lead to or from those sensitive APIs. Addi-

tionally, Shifu et al. [111], analyze the different relationships between function calls and create higher-level semantics which require more efforts for attackers to evade detection.

5.4.4. Registry Operations

The Windows Registry is a hierarchical database that stores low-level settings for the operating system and installed applications. Malware frequently interacts with the Registry to achieve persistence, manipulate configuration settings, or hide its presence [112, 113]. Consequently, monitoring Registry operations during execution is a valuable approach in dynamic malware analysis.

Registry operations refer to actions such as creating, reading, modifying, or deleting registry keys and values. These operations are used by legitimate software to store configuration data but are also commonly exploited by malware for malicious purposes. For example, malware may create or alter keys in the `HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run` path to ensure it runs at system startup, or it may modify security policies to disable system protections . Unlike API or system calls, registry operations provide a higher-level behavioral view that is especially useful for detecting persistence mechanisms and configuration tampering [112, 114].

Registry Operations via Dynamic Analysis

Registry operations are typically captured using sandbox environments or specialized monitoring tools such as `Procmon` [115], `Cuckoo Sandbox`, or Windows event tracing APIs. Dynamic monitoring records events like `RegCreateKey`, `RegSetValue`, and `RegDeleteKey`, which can then be encoded as features for malware classification. Features derived from registry activity include frequency of access, key paths, operation types, and access patterns.

Recent research has emphasized the utility of registry behavior for detecting advanced malware. Aslan and Akin [112] proposed a method focusing on file and registry operations, achieving high accuracy (99.05%) using machine learning algorithms. Similarly, Rosli et al. [116] developed a clustering detection model using K-Means to analyze registry data, resulting in over 90% accuracy. Torres et al. [114] introduced a novel dataset based malware behavioral features including registry. Singh and Singh [117] combined multiple behavioral features, including registry key modifications, with text mining techniques and Shannon entropy calculations, achieving an accuracy

of 99.54% accuracy in malware detection. These findings support the inclusion of registry operations as a key dynamic feature for robust malware detection.

Registry activity is a key behavioral signal in dynamic malware analysis and is represented through several complementary abstractions. Registry-related API call sequences with parameters (e.g., `RegCreateKey`, `RegSetValue`) preserve semantic and contextual information about registry interactions and enable fine-grained behavioral modeling [118, 119, 120]. Operation-count representations summarize the frequency of registry operations such as key creation, deletion, reading, and modification, providing compact and effective statistical features for machine learning classifiers [119, 63]. Registry key and value access patterns capture which paths are accessed and their temporal behavior, supporting clustering and behavioral profiling of malware samples [116, 120, 119]. To reduce noise and improve generalization, studies have utilized normalization or tokenization of registry paths to replace variable components such as randomized identifiers with placeholders [118]. These representations are commonly aggregated into structured feature vectors or integrated into broader behavioral and graph-based models alongside other dynamic indicators, consistently improving detection performance and proving particularly effective for identifying ransomware and other malware families [116, 63].

5.4.5. File System Activities

File system interaction is a core aspect of program behavior and serves as a critical feature in dynamic malware analysis. Malware often manipulates files and directories to achieve persistence, exfiltrate or corrupt data, or hide its components. Monitoring file system activities provides valuable insights into such behaviors, especially those that may not be evident from code alone.

File system activities encompass operations such as file creation, deletion, modification, renaming, and directory traversal. Malware frequently creates hidden or disguised files, overwrites existing system files, or writes payloads to temporary or system-critical directories. These activities may also include interaction with sensitive files, such as logs, registry hives, or system configuration files. Tracking such behavior can reveal attempts to evade detection, manipulate system state, or maintain unauthorized access [121].

File System Monitoring via Dynamic Analysis

Dynamic analysis tools such as `Cuckoo Sandbox`, `Process Monitor`, or `Sysmon` can be used to capture detailed file system activity during execution. Features extracted include the number and types of file operations, file paths accessed, file extensions, and timestamps. These features can then be represented as sequences, frequency histograms, or graph-based models to train machine learning classifiers [121]. Monitoring file access patterns offers a promising approach to identifying malicious activity in real time, even when malware uses obfuscation or cryptography to hide its presence.

Recent studies highlight the discriminative power of file system behavior in malware classification. For instance, Han and Lee (2022) implemented a Kernel-based real-time monitoring structures to track file accesses for for detecting malware activity, making it difficukt for attackers to bypass detection [122]. Other approaches use deception, such as honey files, to detect ransomware by triggering alerts when these decoy files are accessed [123]. Furthermore, l-Hashmi,et al. (2022) showed that file access patterns, when combined with other dynamic features, significantly improve malware detection accuracy [124]. Similarly, Singh and Singh (2020) proposed a hybrid detection model that integrates file system event analysis with other dynamic features to achieve high performance [117]. These findings affirm that file system features are indispensable for revealing real-world malware behaviors during dynamic analysis. Aslan and Akin [125] proposed monitoring file and registry operation areas commonly targeted by malware but rarely by benign software. Using these behavioral features with Random Forest classifiers, the authors achieved high detection accuracy, reinforcing the value of going beyond API calls in dynamic malware analysis.

5.4.6. DLLs (Dynamic Link Libraries)

Dynamic Link Libraries (DLLs) are modular components in Windows operating systems that contain code and data used by multiple programs simultaneously. While they promote code reuse and efficient memory usage, DLLs are frequently exploited by malware to perform malicious activities stealthily.

Malware often leverages DLLs to execute code within the context of legitimate processes, thereby evading detection. Techniques such as DLL injection, sideloading, and hijacking allow malicious code to be loaded into trusted applications. For instance, DLL sideloading involves placing a malicious DLL in a directory where a legitimate application might inadvertently load it,

exploiting the Windows DLL search order [126, 127]. The binary nature of a file, specifically whether it is a Windows Portable Executable (PE), such as a.exe or.dll, is a fundamental attribute that can be used to distinguish potentially malicious files from benign ones. While the PE format itself is not malicious, many types of malware are delivered as PE files due to their ability to contain and execute native machine code. This property, when considered in isolation, may not be a definitive indicator of malicious intent. However, in conjunction with other features such as abnormal API usage, dynamic behavior patterns, or suspicious metadata (e.g., missing signatures, abnormal section sizes), the executable format becomes a strong contributing signal for classifiers and detection systems [128, 129].

Recent literature emphasizes that integrating PE structural features, such as section entropy, number of imported functions, and presence of executable entry points, with dynamic behavioral indicators significantly improves detection accuracy. These combined features are frequently leveraged in modern machine learning pipelines for malware classification [130, 131, 132]. Moreover, identifying whether a DLL or EXE exhibits anomalous executable behaviour (e.g., unexpected process spawning from a DLL) provides critical insights into code misuse or masquerading attempts.

Dynamic Analysis of DLLs

It is straightforward to determine if a file is a DLL by inspecting its PE (Portable Executable) header. Specifically, if the `IMAGE_FILE_DLL` flag (0x2000) is set in the file's header characteristics, the file is identified as a DLL. This can be done using tools like PE viewers or programmatically with libraries such as `pefile` [133].

In static analysis, examining the Import Address Table (IAT) of executables can reveal which DLLs are loaded and which functions are imported. Tools like Dependency Walker and PEStudio facilitate this by providing insights into the dependencies of a binary. However, static analysis might miss dynamically loaded DLLs or those loaded using obfuscated methods [134].

Dynamic analysis, on the other hand, involves monitoring the actual loading and usage of DLLs during program execution. Tools such as Cuckoo Sandbox are widely used for this purpose, providing detailed reports on DLL load events, API calls, and other behavioral artifacts during program execution [135, 136, 137]. System monitoring tools like Sysmon and Elastic Stack can also be used to collect and analyze DLL-related events for real-time detection of malicious activity [138]. These approaches enable comprehensive

behavioral profiling of malware, which is essential for detecting sophisticated threats that evade static approaches.

Analyzing DLL usage patterns is crucial in identifying malicious behavior. Unusual DLL load sequences, DLLs loaded from non-standard locations, rare or rarely-seen DLL names, or high-frequency dynamically-resolved imports

Unusual DLL loading sequences, loading of DLLs from non-standard directories, rarely seen DLLs, or high-frequency dynamically-resolved imports can be indicators of malicious activity. Machine learning models have been developed to classify malware based on DLL usage patterns, and have shown enhanced detection capabilities [139, 140, 141, 142]. Additionally, whether a file is a DLL can be determined dynamically during execution by monitoring specific API calls such as LoadLibrary, GetProcAddress, and LdrLoadDll, which are frequently used to load DLLs at runtime. Dynamic analysis tools and sandboxes track these behaviors to identify the use of malicious DLLs and runtime-loaded libraries. Techniques such as reflective DLL injection, where DLLs are loaded directly into memory without being written to disk, are especially stealthy and can only be detected through runtime monitoring [143, 144]. Such analysis enables detection of evasive behaviors like DLL sideloading and runtime-resolved imports.

5.4.7. Network Activities

Network activity is one of the most critical behavioral indicators in dynamic malware analysis, as many malware families rely on network communication to achieve objectives such as command-and-control (C2) coordination, data exfiltration, lateral movement, and payload delivery. Unlike static indicators, network activities reveal external interactions that directly expose a malware sample's operational intent and infrastructure dependencies.

Network Activity via Dynamic Analysis

Dynamic malware analysis frequently leverages network traffic logs and behavioral profiling to detect and classify malware based on their network activities, which are essential for identifying C2 coordination, data exfiltration, and lateral movement within compromised environments [145]

Raj et al. [146] propose a malware detection framework that applies dynamic packet analysis to extract fine-grained network traffic features such as time-to-live (TTL) and packet length distributions for real-time threat characterization. Their architecture combines ensemble learning models, including Gradient Boosting and Random Forest, with a Multi-Layer Percep-

tron (MLP) meta-classifier to enhance detection performance and robustness against sophisticated network-based malware. The study by Pasquini et al. [147] proposes lightweight packet-level feature representations for characterizing IoT device network behavior directly from raw traffic. It explores shallow neural and compression-based encodings that transform packet headers, payloads, or full packets into compact features for ML classification.

Relevant Feature Combinations

Singh et al. [136] proposed a behavioral malware detection technique by processing runtime features from 16,489 malware samples (collected from VirusShare, TheZoo, VirusTotal, and other repositories) and benign samples (from Windows OS and Contagio). The study applied Single Value Decomposition (SVD) to reduce the dimensionality of PSI string features and computed Shannon entropy over API calls and PSI to model the randomness in malware behavior. Among others, the classifiers tested (KNN, NB, SVM, DT, RF, GB, ADA), API call features consistently achieved the highest detection performance.

To enrich dynamic behavioral analysis, Javed et al. [148] integrated Security Information and Event Management (SIEM) into sandbox analysis, capturing API calls, network traffic, process activity, and registry operations. Performance comparisons among CNN, LSTM, and CNN+LSTM revealed the hybrid CNN+LSTM model as the most effective. Similarly, the study by Rana et al. [149] used supervised (Random Forest, Logistic Regression) and unsupervised models to detect malware in network traffic, demonstrating high accuracy through careful preprocessing. In tackling unknown threats in large-scale systems, Huang et al. [150] extracted both coarse-grained load features and fine-grained statistical features using Text2Vec, enhancing malware detection in network environments. Likewise, Omopintemi et al. [151] assessed RF, SVM, and AdaBoost for classifying malicious traffic, with AdaBoost delivering the highest precision and detection performance. David & Noemí [119] investigated dynamic detection across file types using 19,994 malware and 9,995 benign samples. The study introduced a categorized API call approach, grouping functions into higher-level categories to enhance generalization. Machine learning models using these categorized features outperformed those using individual API calls, achieving 99.37% accuracy with SMAC.

In analyzing API and DLL-based features, Kale et al [152] emphasized the importance of API call sequences in capturing behavioral patterns indicative

of malicious activity. Similarly, Panda et al. [153] developed a knowledge-based model using trusted DLL sequences, represented via TF-IDF and classified with multinomial logistic regression. By comparing suspicious DLL sequences to known profiles using cosine similarity, their method accurately identified trusted, modified, or untrusted processes.

Finally, in optimizing feature combinations for non-network malware classification, ManiArasuSeka et al [154] found that a hybrid feature set combining dynamic API calls, static PSI strings, and DLL features yielded superior performance. Using 7,258 malware samples from MalShare and TheZoo within the Cuckoo Sandbox, the Random Forest model achieved 98.3% accuracy, 0.97 recall, and 0.982 precision, all with low memory and time overhead.

API calls have emerged as one of the most informative dynamic features for malware detection. Existing studies can be broadly grouped into four categories:

(1) Frequency-based features. The frequency of API calls provides strong predictive signals. Studies using statistical selection methods (e.g., Chi-square, Gini) confirmed that specific API categories, such as memory management, significantly improve classification accuracy across models [63, 2, 70].

(2) Sequence-based features. The order and contextual dependencies between API calls capture behavioral intent more effectively than frequency alone. Deep learning methods (CNNs, LSTMs, BiGRUs) and rule-based models have successfully leveraged sequential information to reach accuracies above 95% [64, 68, 66].

(3) Semantic features. Incorporating semantic context into API modeling further enhances robustness. Approaches such as embeddings, topic modeling, and graph neural networks (GNNs) capture latent behavioral patterns beyond raw sequences, improving detection generalization on large-scale datasets [76, 77, 67, 2].

(4) Integrated and hybrid features. Combining API calls with complementary dynamic attributes (e.g., DLL usage, registry operations, network activity, process strings, CTI indicators) consistently outperforms single-feature approaches. Hybrid pipelines achieve superior robustness and accuracy, with several studies reporting detection rates above 98% [78, 79, 125, 148, 154].

In the literature, API calls have been confirmed as foundational features for dynamic malware detection. While raw frequency provides a useful baseline, the most effective approaches enrich API data with sequential dependen-

cies and semantic context, and further integrate them with other behavioral features such as DLLs, registry activity, and network events. Thus, API calls with semantic enrichment and hybrid integration emerge as the most useful feature set for robust and explainable malware detection. In Table 4, we present the various malware feature categories and their common representations in the literature.

6. Analysis Environments, Tools and Framework

Analysis Environment refers to a controlled setup such as sandbox or a virtual machine, where malware is executed and monitored. These environments prevent infection of host systems and allow inspection of system calls, API usage, and file/network interactions. Dynamic analysis can be performed at both the user and kernel levels, with system-wide monitoring frameworks such as Panorama and HookFinder offering comprehensive visibility into process behavior and hooking activities [155, 156]. The malware analysis environment plays a vital role in dynamic malware analysis. It enables analysts to examine the behaviour of malware in controlled environments to identify potential threats and understand their functionality. Provides a platform for the execution of malicious software within tightly monitored and isolated settings to observe and analyse the behaviour of the malware, including file manipulations, network activity, and system changes [157]. The following are the environments, tools, and frameworks essential for dynamic malware analysis.

Table 4: Dynamic malware feature categories and their common representations in the literature

Feature Category	**Common Representations**	**Remark**
API Calls	• Call sequences [64, 66, 72, 65, 75] • Frequency vectors [63, 70, 74, 67] • n-grams [71, 61, 67, 66] • Semantic embeddings [76, 77, 2, 81] • API call graphs [2, 68, 103, 104]	Among API-based dynamic features, sequence- and graph-based representations enriched with semantic information consistently outperform simple frequency-based models, especially against obfuscated and evasive malware [2, 76, 71].
System Calls	• Call sequences [86, 88, 87, 97] • n-grams [86, 88, 96, 92] • Frequency distributions [87, 96, 83] • System call graphs [88, 92, 93]	Frequency-based system call features are efficient but ignore execution order [87, 96], whereas sequence-, n-gram-, and graph-based representations capture temporal and structural behavior, enabling more robust detection of complex and evasive malware at higher computational cost [86, 88, 93].
Function Calls	• Call traces [49, 107, 98] • Function call graphs (FCG) [111, 103, 104, 105] • Sensitive function paths [110, 111, 106]	Function call traces capture runtime execution order but are sensitive to noise and obfuscation [49, 107], whereas function call graphs and sensitive function paths abstract control-flow structure and security-relevant semantics, making them more robust for detecting complex and evasive malware behaviors [111, 110].

Continued on next page

Feature Category	Common Representations	Remark
Registry Activities	• API sequences with parameters [118, 120, 119] • Operation counts [63, 119, 112] • Key/value access patterns [116, 120, 114] • Normalized paths [118, 119]	Operation-count and raw registry access features are lightweight but lose semantic context, whereas API sequences with parameters and normalized key/value paths preserve behavioral intent and improve robustness against obfuscation and ransomware-style persistence mechanisms [118, 119, 116].
File System Operations	• File operation sequences [117, 125] • Access frequencies [123, 122, 125] • Graph representations [121, 103]	Frequency-based file system features are efficient but discard execution context, whereas sequence-, path-, and graph-based representations preserve temporal and structural relationships, enabling robust detection of ransomware and other stealthy malware behaviors [122, 123, 117].
DLL Activities	• Loaded DLL lists [135, 158, 137] • Import tables (IAT) [134, 137] • Runtime load traces [135, 137, 141] • Injection patterns [144, 141]	Static DLL features provide low-cost indicators, while runtime load traces and injection patterns expose stealthy techniques such as DLL sideloading and reflective injection [135, 144, 141].
Network Behavior	• Traffic statistics [145, 149, 146] • Flow features [151, 150, 149] • Temporal patterns [146, 150] • Graph-based communication [145, 147]	Statistical and flow-based network features scale well, while temporal and graph-based representations capture command-and-control and coordinated behaviors [145, 146, 147].
Memory Activities	• Memory dumps [159, 144] • Allocation patterns [84, 144] • Entropy-based features [159, 84]	Memory dumps expose runtime-only artifacts such as unpacked code and fileless malware, while allocation and entropy features reveal in-memory obfuscation [159, 84, 144].

6.1. Sandboxing Systems

Sandboxes provide a virtualized (usually), isolated environment where malware samples can be executed safely, allowing analysts to observe and record their behavior without risking production systems. This approach enables the detection of malicious actions such as file modifications, network communications, process creation, and system calls, as well as the identification of Indicators of Compromise (IOCs) like IP addresses and cryptographic hashes [50, 157, 160].

It is often a cloud-based or on-premises appliance, e.g., Cuckoo Sandbox [161, 135], CAPE Sandbox [161], Any.run [162], Detux[161], CrowdStrike Falcon Sandbox[161], Joe Sandbox [163], VMRay Analyser [163], Drakvuf [164]. They are best suited for Initial triage of suspicious files, High-volume analysis, and Automated IOC generation. Table 5 contains the publicly available sandboxes. Sandboxes are associated with the following:

- Speed and Scalability: Can process a large volume of samples quickly
- Automation: Reduces manual effort in malware analysis
- Baseline Behavior: Provides an initial overview of malware activity
- IOCs Extraction: Easily extracts Indicators of Compromise (IOCs)
- Evasion Techniques: Sophisticated malware can detect sandboxes and alter its behavior or remain dormant
- Limited Interaction: May not capture all behaviors if user interaction is required
- Resource Intensive: Can be resource-heavy for on-premises solutions

6.2. Virtual Machine (VM) Lab (Manual/Interactive)

These are virtualisation technologies that are commonly used to simulate isolated environments where malware execution can be observed [169]. They are controlled and isolated environments built on virtualisation software, which give analysts the leverage to manually execute and interact with malware, utilising specialised tools to monitor and debug it. e.g. VirtualBox, VMware, or Hyper-V. Virtualisation Software(VMware Workstation/Player, VirtualBox), System monitoring tools(Process Monitor, Process Hacker/-Explorer, Wireshark,FakeNet-NG), and Pre-built Toolkits(REMnux, Flare VM) [170, 169]. VMs are associated with the following:

Table 5: Publicly Available Sandboxes For Dynamic Malware Analysis

Sandbox Solution	Type	Key Features / Strengths	Remark
Cuckoo Sandbox [161, 135, 165, 166, 163, 162]	Open-source, on-premises/cloud	Highly customizable dynamic analysis platform; automates malware execution; monitors API calls, file system, registry, and network behaviour; supports multiple OS; extensible via plugins.	Widely used in research and industry; strong community support.
ANY.RUN [162]	Cloud-based (Freemium)	Interactive, real-time analysis, user-friendly interface.	Recognized for their user-friendly interfaces and advanced analysis features but limited offline customisation.
CrowdStrike Falcon Sandbox [163]	Cloud-based (Commercial)	Integrated with threat intelligence, scalable, enterprise-focused.	Commercial solutions known for stealth and enterprise integration.
Joe Sandbox [163]	Commercial, cloud/on-premises	Comprehensive analysis, supports many platforms, advanced evasion resistance.	Recognized for their user-friendly interfaces and advanced analysis features, offering strong support for multiple platforms.
VMRay Analyzer [161]	Commercial, cloud/on-premises	Agentless, high stealth, advanced detection capabilities.	Commercial solutions known for stealth and enterprise integration.
Detux [161]	Open-source	Focused on Linux malware analysis .	Useful for Linux malware; limited compared to multi-OS tools.
CAPEv2 [161]	Open-source (Cuckoo fork)	Focused on unpacking and analyzing malware payloads.	Best open-source tool for analysing packed malware.
Drakvuf [161, 167, 168]	Open-source Open-source, on-premises/cloud (Xen-based)	Agentless, excels at detecting evasive/fileless malware, kernel/user-level tracing	Stands out for its agentless, hypervisor-based approach, which is more effective against evasive and fileless malware, but less mature in tooling and integration; requires technical expertise and Xen hypervisor setup.

- Greater Control in malware analysis, thereby allowing for deep, interactive analysis and manipulation of the environment
- Customisation, allowing analysis to be tailored with specific OS versions, software, and configurations
- Snapshotting enabling easy reverting to a clean state after each analysis
- Evasion testing can be configured to make malware detection of the virualisation more difficult

Just like the automated sandboxes, some sophisticated malware are capable of detecting virtualized environments, thereby hiding or modifying their behavior to evade detection. However, since an analyst has more control over a manually designed virtual machine, it is easier to configure a dedicated virtual machine to make detection of the virtualization more difficult.

6.3. Physical Machine Lab

It is a dedicated, isolated physical machine (or set of machines) that is used for malware analysis. It is often air-gapped or on a strictly controlled network. e.g System Monitoring and Analysis(Process Monitor, Process Hacker, and Wireshark), Debugger(WinDbg,x64dbg). This tool is best suited for analysing highly evasive malware, Malware interacting with specific hardware, Rootkit and bootkit analysis, and it is a *"Last resort"* analysis for difficult samples [26]. It offers the following advantages.

- Evasion Resistance: Malware finds it much harder to detect a physical machine environment compared to virtualised ones
- True Behavior: Provides the most accurate representation of malware behaviour in a real system
- Hardware Interaction: Essential for analysing malware that interacts directly with hardware (e.g., firmware implants, rootkits)

6.4. Emulation

This is a malware analysis tool that simulates the CPU and other hardware components in software, allowing the analyst to execute and observe malware instructions at a very low level without running a full operating system [171]. e.g QEMU, Unicorn Engine, and Qiling Framework. It is best

suited for analysing highly obfuscated or evasive malware, for understanding low-level execution flow, and Cross-architecture analysis [171]. It ooffers the following advantages:

- Deep Visibility: Provides granular insight into instruction execution and memory access
- High Evasion Resistance: Extremely difficult for malware to detect the emulation environment, and
- No OS Dependencies: Can analyse samples designed for different operating systems or architectures

These components are essential for comprehensive dynamic malware analysis, facilitating accurate threat detection and behaviour profiling.

6.4.1. Dynamic Analysis Tools

API calls are vital behavioral indicators in malware analysis, revealing how malware interacts with system resources. Tools for extracting these calls are generally classified into static, dynamic, and hybrid approaches. Dynamic tools, such as API Monitor and Process Monitor, capture API calls at runtime by hooking into executing processes [172, 173]. Cuckoo Sandbox automates dynamic analysis, logging API call traces along with file, registry, and network activities [91]. Tools like Frida and Intel PIN allow for scriptable instrumentation and custom hooking of APIs, offering deeper control over extraction [174, 175]. Static tools like IDA Pro, Ghidra, and PEStudio extract API references by analyzing binary imports and control flows without execution [176, 177]. While safer, they may miss obfuscated or dynamically resolved calls. For machine learning, API sequences are often transformed into n-grams or behavioral profiles using tools like Malheur [178]. Hybrid approaches such as DroidDissector [179] combining static and dynamic techniques, are often most effective against modern evasive malware.

Dynamic malware analysis also requires specialized tools to extract key system features such as dynamic-link libraries (DLLs), process-level information, network activities, and registry modifications. These tools help analysts uncover malware behaviors and develop forensic indicators.

1. DLL Extraction
    - PE-sieve: Detects and dumps memory-injected or replaced DLLs[180].

- Process Hacker / Process Explorer: Visual tools for inspecting DLLs loaded within running processes[158].
- Volatility (dlllist, malfind): Memory forensic plugins to locate and extract injected DLLs.

2. Process and System Information (PSI)

   - ProcMon: Logs file, registry, and process/thread activity in real time[173].
   - Sysmon: Captures persistent logs of process creation, network connections, and hashes[181].

3. Network Traffic

   - Wireshark: A packet analyzer for live traffic capture and inspection[182].
   - FakeNet-NG: Emulates network services to analyze malware communication behaviour[183].

4. Registry Feature Extraction

   - Regshot: Captures and compares registry changes before and after execution[184, 185].
   - Autoruns: Identifies auto-start registry entries often used by malware[186].

Table 6: Feature Extraction Tools Used in Dynamic Malware Analysis

Tools	**Platform**	**Extraction Method**	**Description**
Intel PIN[109]	Windows, Linux (x86/x64)	Dynamic Binary Instrumentation (DBI)	Precise, storage-heavy; ideal for control-flow and opcode features.
DynamoRIO [108]	Windows, Linux, macOS	DBI / Code cache instrumentation	Lightweight runtime metrics; alternative to PIN.
Frida [187]	Windows, Linux, macOS, Android, iOS	Dynamic API hooking via injected agent	Scriptable and lightweight; good for runtime API tracing.
Apicula [188]	Windows	API hooking through DLL injection	GUI-based; ideal for manual feature inspection.
Microsoft Detours [189]	Windows	Inline function interception	Used to build custom API tracers.
x64dbg + ScyllaHide [190]	Windows	Debugger-based hooking	Scriptable; ScyllaHide bypasses anti-debugging.
strace [89]	Linux	ptrace-based syscall interception	Core Linux syscall-level extraction tool.
ltrace [191]	Linux	Library-call tracing (libc)	Complements strace for user-space calls.
Sysmon[90]	Windows	Kernel-mode logging via driver	Generates structured telemetry for feature extraction.
Procmon[173]	Windows	Kernel + user event capture	Detailed event-level behavior monitoring.
Volatility 3[192]	Windows, Linux, macOS	Post-acquisition memory forensics	Core framework for feature extraction from memory dumps.
Rekall[193]	Windows, Linux	Memory forensics	Alternative to Volatility.

Tools	Platform	Extraction Method	Description
LiME[194]	Linux	Kernel module for memory acquisition	Feeds Volatility/Rekall for analysis.
DumpIt[195]	Windows / Linux	Memory acquisition	Captures live memory for post-analysis.
LibVMI[196]	KVM / Xen hypervisors	Virtual Machine Introspection (VMI)	External monitoring; avoids detection by malware.
Zeek[197, 198]	Linux / Network sensors	PCAP parsing	Rich behavioral network features for C2 detection.
Suricata[199, 197]	Cross-platform	IDS / protocol parser	Structured output suitable for ML pipelines.
tshark / tcpdump[200, 201]	Cross-platform	Packet capture and parsing	Low-level capture; feeds Zeek/Suricata.
JA3 / JA3S[202]	Any	TLS fingerprinting	Detects encrypted command-and-control behavior.
FIM[203]	Linux	Filesystem event monitoring	Tracks created or modified files during execution.

6.5. Dynamic Malware Analysis Workflow

This section outlines a structured framework and step-by-step workflow to perform dynamic malware analysis, highlighting key phases such as environment setup, behavioral monitoring, and data interpretation. Each step is supported by specific tools and resources, such as sandboxes, monitoring utilities, and malware datasets, which are detailed in the corresponding section and tables for tools and datasets in this paper. The aim is to provide a practical and reproducible guide for researchers and practitioners to effectively analyze and understand malware behavior in real time. The workflow in Figure 8 illustrates the sequence of decisions and actions required to safely execute and analyse malware samples within either an automated sandbox or a dedicated virtual machine environment.

Analysis Objectives and Scope: Establishing a clear objective and scope is the first step to conducting a dynamic analysis. Here, the analyst clearly defines the type of behavioral information (i.e., specific feature sets) targeted (e.g., API calls). And the scope, such as the malware domain of interest (e.g. Windows, Android, PDF, etc), and what limitations must be respected, such as the absence of internet connectivity or the need for controlled network simulation. Defining the scope at this stage ensures that the later choice of analysis environment, monitoring depth, dataset and execution strategy are aligned with the intended research or operational outcomes.

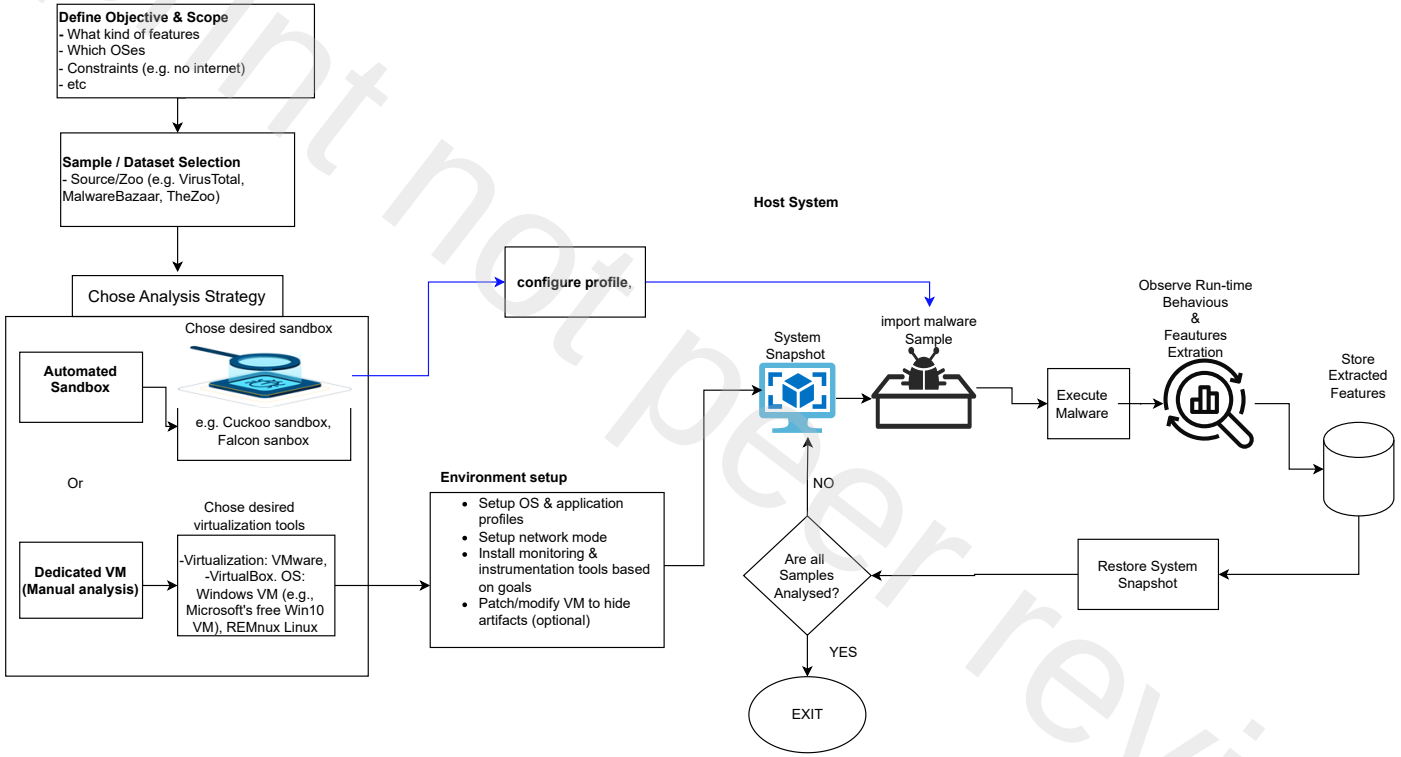

Figure 8: Steps for dynamic malware analysis

Sample/Dataset Selection: Based on the objective, the analyst selects the malware samples or dataset to be examined. Samples may originate from public malware repositories such as VirusTotal, MalwareBazaar, or TheZoo, or from organisational incident reports (see Table 3 for a comprehensive list of public repositories of live samples). Once samples are selected, they are securely transferred to the analysis host, where preparation and profiling take place. Researchers commonly label malware samples using a multi-antivirus consensus approach, where each sample is scanned with several reputable engines such as Kaspersky, Symantec, McAfee, Bitdefender, ESET, Avast, and Windows Defender. A sample is considered malicious only when multiple engines often a majority or a minimum threshold such as three to five of the engines detect it as malware, ensuring more reliable and noise-reduced ground-truth labels.

Choosing an Analysis Strategy After defining the objective and selecting the desired samples for analysis, a key decision involves selecting the analysis strategy. The analyst may choose an automated sandbox, such as Cuckoo Sandbox or Falcon Sandbox, when high throughput. Alternatively, a dedicated virtual machine may be selected for manual analysis when deeper, more interactive investigation is necessary. Dedicated VM setups often rely on VMware, VirtualBox, or specialised distributions such as REMnux for controlled malware execution. This choice determines how much control, interaction, and instrumentation the analyst will have during the execution phase. For a choice of employing an automated sandbox, Table 5 presents a list of sandboxes. It important to note that each sandbox has its strengths and weaknesses and is capable of detecting different features, or it operates at different levels of the operating system (user-level vs. kernel-level). Therefore, it is advisable to use a combination of multiple sandboxes during analysis.

Environment Setup Once the analysis strategy is selected, the environment is prepared accordingly. In both automated sandboxes and dedicated virtual machines, this includes configuring the operating system image, installing any required applications to emulate realistic host behaviour, and setting the desired network mode, ranging from full isolation to controlled internet access. Automated sandboxes perform much of this configuration implicitly, as they rely on predefined, reusable analysis environments that require minimal analyst intervention. In contrast, dedicated manual VMs require the analyst to explicitly install monitoring and instrumentation tools, such as system event loggers, API tracers, and network capture utilities and to tune

the environment to the needs of the experiment. Table 6 presents diverse monitoring and instrumentation tools. In manual settings, the analyst may also take additional steps to disguise virtualization artefacts to reduce the likelihood of sandbox evasion by the malware.

Profile Configuration and Snapshot Creation: Before any malware sample is introduced, the analyst configures the profile or VM state according to the selected strategy. In an automated sandbox, this involves selecting the appropriate OS profile and analysis parameters. In a manual VM, the analyst configures system settings, prepares the sample execution environment, and then creates a system snapshot. This snapshot represents a clean state that can later be restored after execution to prevent contamination and ensure repeatability.

Sample Import and Execution: The prepared malware sample is introduced into the analysis environment using different mechanisms depending on the chosen strategy. Automated sandboxes handle sample import and execution automatically, typically through a submission interface or API that places the sample into a preconfigured virtual environment, launches it, and initiates monitoring without further analyst intervention. In contrast, dedicated manual VMs require the analyst to transfer the sample into the guest system, such as via shared folders, ISO images, or controlled network channels and manually initiate its execution. Once the sample is running, the environment records its behaviour, enabling the observation of process creation, file and registry modification, network activity, and any attempts to detect or evade the analysis platform.

Behaviour Observation and Feature Extraction During execution, runtime behaviour is continuously observed and logged. The monitoring tools record system interactions and operational traces, which are aggregated to support subsequent feature extraction. These features may include changes to the filesystem, registry alterations, spawned processes, injected code, or network requests. The extracted features are stored for further analysis, classification, or machine learning experiments.

Environment Restoration and Sample Iteration After each execution, the system snapshot created earlier is restored to return the environment to a clean and uncompromised state. This ensures that no residual artefacts persist and allows the next sample to be evaluated under identical conditions. The workflow proceeds iteratively until all selected samples have been analysed, after which the process terminates cleanly.

6.6. Open Challenges in Dynamic Malware Analysis

Despite its effectiveness in revealing runtime behavior, dynamic malware analysis faces several open challenges that limit its reliability and scalability. A major issue arises from malware techniques explicitly designed to evade dynamic analysis. Similar to static evasion methods such as obfuscation or packing, modern malware increasingly incorporates mechanisms to detect sandboxed or virtualized environments. These include checks for virtual machine artifacts, monitoring of user interaction patterns, inspection of system configurations, and validation of real network connectivity. Malware may also probe for specific IP addresses, domains, or external services before activating its payload [204, 26, 205]. Or-Meir et al. [26] survey modern dynamic malware analysis and emphasize that, although more robust than static analysis, no single dynamic tool can cover all behaviors, and many techniques are vulnerable to anti-analysis and evasion mechanisms, including virtualization/sandbox detection and limited behavior coverage during short runs.

Dynamic analysis inherently suffers from limited execution coverage: many behaviors appear only under rare conditions, particular execution paths, or with multi-stage payloads and delayed triggers (e.g., time-based logic bombs), so a bounded analysis window may miss such behaviours. Galloro et al. [205] stress that evasive techniques such as delayed execution and stalling are used to slow or postpone malicious behavior, exploiting the fact that sandboxes typically run samples for limited time for scalability, thereby keeping payloads dormant during analysis.

Furthermore, many dynamic systems (emulators, heavy instrumentation) incur significant performance overhead, which is problematic for large-scale and real-time deployment; they frame efficiency, transparency, and analysis quality as conflicting design goals for dynamic analysis [206]

7. Conclusion

This survey has provided a comprehensive and structured examination of state-of-the-art dynamic malware features and their role in modern malware detection systems. By synthesizing findings across recent literature, the study clarifies conceptual ambiguities in existing taxonomies, highlights inconsistencies in feature definitions, and brings forward a unified, improved taxonomy for dynamic malware analysis. The review demonstrates that dynamic features, spanning API calls, system calls, registry updates, file system

activities, network behavior, DLL injections, and memory artifacts, remain indispensable for identifying evasive and polymorphic malware that eludes purely static approaches.

Furthermore, this work contributes a practical, end-to-end framework for conducting dynamic malware analysis, including curated tools, environments, and a comprehensive mapping of publicly available datasets that support reproducibility and benchmarking. This framework fills a critical gap by offering researchers a structured, executable workflow rather than fragmented, tool-oriented guidance often seen in prior studies. Our analysis also illustrates the increasing integration of machine learning in feature engineering and detection pipelines, while emphasizing how detection accuracy and robustness strongly depend on the quality, granularity, and consistency of extracted dynamic features.

Overall, this survey consolidates the current state of knowledge, exposes prevailing challenges, and offers clarity in an area where contradictory definitions and incomplete feature coverage have hindered progress. It serves as both a conceptual foundation and practical reference point for future work in malware behavior analysis and intelligent detection systems.

CRediT authorship contribution statement

Monday Onoja, Peter Anthony, Zekeri Adams, and Kefas Rimamnuskeb Galadima: Conceptualization, Methodology, Investigation, Formal analysis, Data curation, Writing – original draft, Writing – review & editing, Visualization, Validation, Resources. **Martin Homola and Štefan Balogh:** Conceptualization, Methodology, Validation, Writing – review & editing, Resources, Supervision, Project administration, Funding acquisition. **Ján Kľuka and Roderik Ploszek:** Methodology, Formal analysis, Investigation, Data curation, Writing – review & editing, Visualization. **Temidayo Oluwatosin Omotehinwa and Abayomi Joshua Jegede:** Conceptualization, Methodology, Writing – review & editing, Resources.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

This work is funded by the EU NextGenerationEU through the Recovery and Resilience Plan for Slovakia under the project No. 09I05-03-V02-00064.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this manuscript, the authors used ChatGPT to assist in refining the writing. Following the use of this tool, the authors carefully reviewed, edited, and validated the content and take full responsibility for the accuracy, originality, and integrity of the published work.

References

[1] C. Ventures, Cybercrime to cost the world $10.5 trillion annually by 2025, accessed: 20 January 2025 (2020).
URL https://cybersecurityventures.com/hackerpocalypse-cybercrime-report-2016/

[2] C. Li, Z. Cheng, H. Zhu, L. Wang, Q. Lv, Y. Wang, N. Li, D. Sun, Dmalnet: Dynamic malware analysis based on api feature engineering and graph learning, Computers Security 122 (2022). doi:10.1016/j.cose.2022.102872.

[3] A.-T. Institute, Malware statistics & trends report, accessed: 20 January 2026 (2026).
URL https://www.av-test.org/en/statistics/malware/

[4] A.-T. Institute, About malware and pua | av-test, accessed: 20 January 2026 (2025).
URL https://portal.av-atlas.org/malware

[5] Statcounter Global Stats, Desktop operating system market share worldwide, accessed: 20 January 2026 (2026).
URL https://gs.statcounter.com/os-market-share/desktop/worldwide/

[6] J. S. Sraw, K. Kumar, Using static and dynamic malware features to perform malware ascription, ECS Transactions 107 (1) (2022) 3187. doi:https://dx.doi.org/10.1149/10701.3187ecst.

[7] M. Onoja, A. Jegede, J. Mazadu, G. Aimufua, A. Oyedele, K. Olibodum, Exploring the effectiveness and efficiency of lightgbm algorithm for windows malware detection, in: 2022 5th Information Technology for Education and Development (ITED), 2022, pp. 1–6. doi:10.1109/ITED56637.2022.10051488.

[8] M. Onoja, A. Jegede, N. Blamah, O. Abimbola, T. O. Omotehinwa, Eemds: Efficient and effective malware detection system with hybrid model based on xceptioncnn and lightgbm algorithm, Journal of Computing and Social Informatics 1 (2) (2022) 42–57. doi:10.33736/jcsi.4739.2022.

[9] S. Naik, A. Dessai, Malware classification approaches using machine learning techniques: A review, in: 2021 5th International Conference on Electrical, Electronics, Communication, Computer Technologies and Optimization Techniques (ICEECCOT), 2021, pp. 111–117. doi:10.1109/ICEECCOT52851.2021.9708041.

[10] Light up that droid! on the effectiveness of static analysis features against app obfuscation for android malware detection, Journal of Network and Computer Applications 235 (2025) 104094. doi:https://doi.org/10.1016/j.jnca.2024.104094.
URL https://www.sciencedirect.com/science/article/pii/S1084804524002716

[11] J. Geng, J. Wang, Z. Fang, Y. Zhou, D. Wu, W. Ge, A survey of strategy-driven evasion methods for pe malware: Transformation, concealment, and attack, Computers Security 137 (2024) 103595. doi:https://doi.org/10.1016/j.cose.2023.103595.

[12] B. Xu, Y. Li, X. Yu, Malware detection based on static and dynamic features analysis, in: Machine Learning for Cyber Security, Vol. 12486 of Lecture Notes in Computer Science, Springer, 2020, pp. 111–124. doi:10.1007/978-3-030-62223-7_10.
URL https://doi.org/10.1007/978-3-030-62223-7_10

[13] McAfee, What is malware?, retrieved from https://www.mcafee.com/enterprise/en-us/security-awareness/malware.html (n.d.).

[14] Kaspersky Lab, What is malware?, retrieved from https://www.kaspersky.com/resource-center/definitions/malware (n.d.).

[15] D. Ucci, L. Aniello, R. Baldoni, A survey on malware detection techniques, Computers Security 57 (2020) 97–129. doi:10.1016/j.cose.2015.11.001.

[16] Y. Choi, S. H. Kim, J. S. Lee, Kernel-level malware detection using behavior profiling, in: 2013 IEEE Conference on Communications and Network Security (CNS), IEEE, 2013, pp. 1–9. doi:10.1109/CNS.2013.6614228.

[17] R. Sihwail, K. B. Omar, K. A. Z. Ariffin, A survey on malware analysis techniques: Static, dynamic, hybrid and memory analysis, International Journal on Advanced Science, Engineering and Information Technology (2018). URL https://api.semanticscholar.org/CorpusID:54017419

[18] C. Kolbitsch, P. M. Comparetti, C. Kruegel, F. Zhou, X. Wang, Effective and efficient malware detection at the end host, in: USENIX Security Symposium, USENIX Association, 2009, pp. 351–366. doi:10.1513/97-88239.

[19] G. Bilar, Analyzing malware: A case study, IEEE Security & Privacy 5 (6) (2007) 75–78. doi:10.1109/MSP.2007.103.

[20] M. Al-Janabi, A. M. Altamimi, A comparative analysis of machine learning techniques for classification and detection of malware, in: 21st International Arab Conference on Information Technology (ACIT), 2020, pp. 1–9. doi:10.1109/ACIT50332.2020.9300081.

[21] R. Sihwail, K. B. Omar, K. A. Z. Ariffin, A survey on malware analysis techniques: Static, dynamic, hybrid, and memory analysis, International Journal on Advanced Science, Engineering and Information Technology 8 (4-2) (2020) 1662–1671. doi:10.18517/ijaseit.8.4-2.6827.

[22] M. Egele, T. Scholte, E. Kirda, C. Kruegel, A survey on automated dynamic malware-analysis techniques and tools, ACM Computing Surveys (CSUR) 44 (2) (2022). doi:10.1145/2203376.2203384.

[23] S. Naik, A. Dessai, Malware classification approaches using machine learning techniques: A review, 5th International Conference on Electrical, Electronics, Communication, Computer Technologies and Optimization Techniques (ICEECCOT) (2021) 111–117doi:10.1109/ICEECCOT52851.2021.9708041.

[24] M. Christodorescu, S. Jha, Semantics-aware malware detection, in: Proceedings of the 10th ACM Conference on Computer and Communications Security, 2021, pp. 32–41. doi:10.1145/948109.948121.

[25] F. Mira, A review paper of malware detection using api call sequences, 2nd International Conference on Computer Applications Information Security (ICCAIS) (2019) 1–6doi:10.1109/CAIS.2019.8769564.

[26] O. Or-Meir, N. Nissim, Y. Elovici, L. Rokach, Dynamic malware analysis in the modern era—a state of the art survey, ACM Computing Surveys 52 (5) (2019) 1–48. doi:10.1145/3329786.
URL https://doi.org/10.1145/3329786

[27] M. G. Gaber, M. Ahmed, H. Janicke, Malware detection with artificial intelligence: A systematic literature review, ACM Comput. Surv. 56 (6) (Jan. 2024). doi:10.1145/3638552.

[28] N. Pachhala, S. Jothilakshmi, B. P. Battula, A comprehensive survey on identification of malware types and malware classification using machine learning techniques, in: 2nd International Conference on Smart Electronics and Communication (ICOSEC), 2021, pp. 1207–1214. doi:10.1109/ICOSEC51865.2021.9591763.

[29] S. S. Chakkaravarthy, D. Sangeetha, V. Vaidehi, A survey on malware analysis and mitigation techniques, Computer Science Review 32 (2019) 1–23. doi:10.1016/j.cosrev.2019.01.002.
URL https://doi.org/10.1016/j.cosrev.2019.01.002

[30] M. A. Jerlin, A dynamic Malware Analysis for Windows Platform - a survey, Indian Journal of Science and Technology 8 (1) (2015) 1–5. doi:10.17485/ijst/2015/v8i26/81172.
URL https://doi.org/10.17485/ijst/2015/v8i26/81172

[31] N. Pachhala, S. Jothilakshmi, B. P. Battula, A comprehensive survey on identification of malware types and malware classification using machine learning techniques, in: 2021 2nd International Conference on Smart Electronics and Communication (ICOSEC), 2021, pp. 1207–1214. doi:10.1109/ICOSEC51865.2021.9591763.

[32] M. Al-Janabi, A. M. Altamimi, A comparative analysis of machine learning techniques for classification and detection of malware, in: 2020 21st International Arab Conference on Information Technology (ACIT), 2020, pp. 1–9. doi:10.1109/ACIT50332.2020.9300081.

[33] S. J. Irzal Ismail, Hendrawan, B. Rahardjo, A survey on malware detection technology and future trends, in: 2020 14th International Conference on Telecommunication Systems, Services, and Applications (TSSA, 2020, pp. 1–6. doi:10.1109/TSSA51342.2020.9310841.

[34] A. Cletus, A. A. Opoku, B. A. Weyori, An Evaluation of Current Malware Trends and Defense Techniques: A Scoping Review with Empirical Case

Studies, Journal of Advances in Information Technology (2024) 649–671doi:10.12720/jait.15.5.649-671.

[35] N. Kaur, A. Kumar, A complete dynamic malware analysis, International Journal of Computer Applications 135 (4) (2016) 20–25. doi:10.5120/ijca2016908283.
URL https://doi.org/10.5120/ijca2016908283

[36] R. Sihwail, K. Omar, K. A. Z. Ariffin, A survey on malware analysis techniques: static, dynamic, hybrid and memory analysis, International Journal on Advanced Science Engineering and Information Technology 8 (4-2) (2018) 1662–1671. doi:10.18517/ijaseit.8.4-2.6827.

[37] M. G. Gaber, M. Ahmed, H. Janicke, Malware Detection with Artificial Intelligence: A Systematic Literature Review, ACM Computing Surveys 56 (6) (2023) 1–33. doi:10.1145/3638552.
URL https://doi.org/10.1145/3638552

[38] D. Ucci, L. Aniello, R. Baldoni, Survey of machine learning techniques for malware analysis, Computers Security 81 (2018) 123–147. doi:10.1016/j.cose.2018.11.001.
URL https://doi.org/10.1016/j.cose.2018.11.001

[39] P. Shijo, A. Salim, Integrated static and dynamic analysis for malware detection, Procedia Computer Science 46 (2015) 804–811. doi:10.1016/J.PROCS.2015.02.149.

[40] M. Ijaz, M. Durad, M. Ismail, Static and dynamic malware analysis using machine learning, in: 2019 16th International Bhurban Conference on Applied Sciences and Technology (IBCAST), 2019, pp. 687–691. doi:10.1109/IBCAST.2019.8667136.

[41] B. Dondogmegd, B. Usukhbayr, J. Nyamjav, A malware analysis using static and dynamic techniques, Science Journal of Business and Management 5 (2015) 20. doi:10.11648/J.SJBM.S.2016050101.15.

[42] A. Damodaran, F. D. Troia, C. Visaggio, T. Austin, M. Stamp, A comparison of static, dynamic, and hybrid analysis for malware detection, Journal of Computer Virology and Hacking Techniques 13 (2015) 1–12. doi:10.1007/s11416-015-0261-z.

[43] K. Khalda, D. K. Wibowo, Malware behavior analysis using static and dynamic analysis approaches, Jurnal Sains, Nalar, dan Aplikasi Teknologi Informasi (2025). doi:10.20885/snati.v4.i1.1.

[44] P. Maniriho, A. Mahmood, M. Chowdhury, A study on malicious software behaviour analysis and detection techniques: Taxonomy, current trends and challenges, Future Generation Computer Systems 130 (2021) 1–18. doi:10.1016/j.future.2021.11.030.

[45] S. O'Shaughnessy, S. Sheridan, Image-based malware classification hybrid framework based on space-filling curves, Computers & Security 116 (2022) 102660. doi:10.1016/j.cose.2022.102660.

[46] C. Ding, N. Luktarhan, B. Lu, W. Zhang, A hybrid analysis-based approach to android malware family classification, Entropy 23 (2021). doi:10.3390/e23081009.

[47] W. Han, J. Xue, Y. Wang, L. Huang, Z. Kong, L. Mao, Maldae: Detecting and explaining malware based on correlation and fusion of static and dynamic characteristics, Computers & Security 83 (2019) 208–233. doi:10.1016/J.COSE.2019.02.007.

[48] H. Wang, W. Zhang, H. He, You are what the permissions told me! android malware detection based on hybrid tactics, Journal of Information Security and Applications 66 (2022) 103159. doi:10.1016/j.jisa.2022.103159.

[49] S. Mohammed A. F., M. F. Marhusin, R. Sulaiman, Instrumenting api hooking for a realtime dynamic analysis, in: 2019 International Conference on Cybersecurity (ICoCSec), 2019, pp. 49–52. doi:10.1109/ICoCSec47621.2019.8971017.

[50] R. R. R, N. S, S. R, T. S, Malware analysis using sandbox, Tech. Rep. 4708146, SSRN, available at SSRN (January 27 2024). doi:10.2139/ssrn.4708146.
URL https://ssrn.com/abstract=4708146

[51] N. Owoh, J. Adejoh, S. Hosseinzadeh, M. Ashawa, J. Osamor, A. Qureshi, Malware detection based on api call sequence analysis: A gated recurrent unit–generative adversarial network model approach, Future Internet 16 (10) (2024). doi:10.3390/fi16100369.
URL https://www.mdpi.com/1999-5903/16/10/369

[52] T. Jiang, L. Cui, Z. Lin, F. Lu, A survey of malware classification methods based on data flow graph, in: Proceedings of the International Conference of Pioneering Computer Scientists, Engineers and Educators (ICPCSEE 2022), Vol. 1628 of Communications in Computer and Information Science, Springer, 2022, pp. 80–93. doi:10.1007/978-981-19-5194-7_7.

[53] D. Demırcı, N. şahın, M. şirlancis, C. Acarturk, Static malware detection using stacked bilstm and gpt-2, IEEE Access 10 (2022) 58488–58502. doi:10.1109/ACCESS.2022.3179384.

[54] L. Akhmetzyanova, C. Cremers, L. Garratt, S. Smyshlyaev, N. Sullivan, Limiting the impact of unreliable randomness in deployed security protocols, in: 2020 IEEE 33rd Computer Security Foundations Symposium (CSF), 2020, pp. 277–287. doi:10.1109/CSF49147.2020.00027.

[55] B. Cheng, J. Ming, E. A. Leal, H. Zhang, J. Fu, G. Peng, J.-Y. Marion, Obfuscation-Resilient executable payload extraction from packed malware, in: 30th USENIX Security Symposium (USENIX Security 21), USENIX Association, 2021, pp. 3451–3468.
URL https://www.usenix.org/conference/usenixsecurity21/presentation/cheng-binlin

[56] V. Kotov, M. Wojnowicz, Towards generic deobfuscation of windows API calls, CoRR abs/1802.04466 (2018). arXiv:1802.04466.
URL http://arxiv.org/abs/1802.04466

[57] V. Arceri, I. Mastroeni, Analyzing dynamic code: A sound abstract interpreter for evil eval, ACM Trans. Priv. Secur. 24 (2) (Jan. 2021). doi:10.1145/3426470.
URL https://doi.org/10.1145/3426470

[58] G. Antal, P. Hegedűs, Z. Herczeg, G. Lóki, R. Ferenc, Is javascript call graph extraction solved yet? a comparative study of static and dynamic tools, IEEE Access 11 (2023) 25266–25284. doi:10.1109/ACCESS.2023.3255984.

[59] J. Samhi, R. Just, M. D. Ernst, T. F. Bissyandé, J. Klein, Resolving conditional implicit calls to improve static and dynamic analysis in android apps, ACM Trans. Softw. Eng. Methodol.Just Accepted (Apr. 2025). doi:10.1145/3729168.
URL https://doi.org/10.1145/3729168

[60] M. D'Onghia, M. Salvadore, B. M. Nespoli, M. Carminati, M. Polino, S. Zanero, Apícula: Static detection of api calls in generic streams

of bytes, Computers Security 119 (2022) 102775. doi:https://doi.org/10.1016/j.cose.2022.102775.
URL https://www.sciencedirect.com/science/article/pii/S0167404822001705

[61] M. Nawaz, P. Fournier-Viger, M. Nawaz, G. Chen, Y. Wu, Metamorphic malware behavior analysis using sequential pattern mining (2021) 90–103doi:10.1007/978-3-030-93733-1_6.

[62] P. Wu, M. Gao, F. Sun, X. Wang, L. X. Pan, Multi-perspective api call sequence behavior analysis and fusion for malware classification, Comput. Secur. 148 (2024) 104177. doi:10.1016/j.cose.2024.104177.

[63] D. Z. Syeda, M. N. Asghar, Dynamic malware classification and api categorisation of windows portable executable files using machine learning, Applied Sciences 14 (3) (2024). doi:10.3390/app14031015.
URL https://www.mdpi.com/2076-3417/14/3/1015

[64] G. D'Angelo, M. Ficco, F. Palmieri, Association rule-based malware classification using common subsequences of api calls, Applied Soft Computing 105 (2021) 107234. doi:https://doi.org/10.1016/j.asoc.2021.107234.

[65] Z. Wu, J. Zhang, L. Kou, A model for malware detection method based on api call sequence clustering, in: 2022 9th International Conference on Dependable Systems and Their Applications (DSA), 2022, pp. 1049–1050. doi:10.1109/DSA56465.2022.00157.

[66] Prachi., N. Dabas, P. Sharma, Malanalyser: An effective and efficient windows malware detection method based on api call sequences, Expert Systems with Applications 230 (2023) 120756. doi:https://doi.org/10.1016/j.eswa.2023.120756.

[67] V. Voronin, A. Morozov, Analyzing api sequences for malware monitoring using machine learning, in: 2021 3rd International Conference on Control Systems, Mathematical Modeling, Automation and Energy Efficiency (SUMMA), 2021, pp. 519–522. doi:10.1109/SUMMA53307.2021.9632005.

[68] D. Wu, P. Guo, P. Wang, Malware detection based on cascading xgboost and cost sensitive, in: 2020 International Conference on Computer Communication and Network Security (CCNS), 2020, pp. 201–205. doi:10.1109/CCNS50731.2020.00051.

[69] Z. Zhang, P. Qi, W. Wang, Dynamic malware analysis with feature engineering and feature learning, Proceedings of the AAAI Conference on Artificial Intelligence 34 (01) (2020) 1210–1217. doi:10.1609/aaai.v34i01.5474.
URL https://ojs.aaai.org/index.php/AAAI/article/view/5474

[70] D. Z. Syeda, M. N. Asghar, Enhancing malware classification: A comparison of filter and embedded feature selection with a novel dataset, in: 2023 Cyber Research Conference - Ireland (Cyber-RCI), 2023, pp. 1–8. doi:10.1109/Cyber-RCI59474.2023.10671445.

[71] G. D'Angelo, M. Ficco, F. Palmieri, Association rule-based malware classification using common subsequences of api calls, Appl. Soft Comput. 105 (2021) 107234. doi:10.1016/J.ASOC.2021.107234.

[72] P. Maniriho, A. N. Mahmood, M. J. M. Chowdhury, Api-maldetect: Automated malware detection framework for windows based on api calls and deep learning techniques, Journal of Network and Computer Applications 218 (2023) 103704. doi:https://doi.org/10.1016/j.jnca.2023.103704.

[73] E. Amer, I. Zelinka, A dynamic windows malware detection and prediction method based on contextual understanding of api call sequence, Computers Security 92 (2020) 101760. doi:https://doi.org/10.1016/j.cose.2020.101760.
URL https://www.sciencedirect.com/science/article/pii/S0167404820300444

[74] T. Tomiyama, Q. Wu, A. Kanai, Dynamic analysis for malware detection using api calls and memory usage with machine learning approach, in: 2023 IEEE 12th Global Conference on Consumer Electronics (GCCE), 2023, pp. 475–477. doi:10.1109/GCCE59613.2023.10315559.

[75] S. Zhang, M. Gao, L. Wang, S. Xu, W. Shao, R. Kuang, A malware-detection method using deep learning to fully extract api sequence features, Electronics 14 (1) (2025) 167. doi:10.3390/electronics14010167.
URL https://www.mdpi.com/2079-9292/14/1/167

[76] C. Li, Q. Lv, N. Li, Y. Wang, D. Sun, Y. Qiao, A novel deep framework for dynamic malware detection based on api sequence intrinsic features, Computers Security 116 (2022) 102686. doi:https://doi.org/10.1016/j.cose.2022.102686.
URL https://www.sciencedirect.com/science/article/pii/S0167404822000840

[77] S. Zhang, J. Wu, M. Zhang, W. Yang, Dynamic malware analysis based on api sequence semantic fusion, Applied Sciences 13 (11) (2023). doi:10.3390/app13116526.

[78] Namita, Prachi, P. Sharma, Windows malware detection using machine learning and tf-idf enriched api calls information, in: 2022 Second International Conference on Computer Science, Engineering and Applications (ICCSEA), 2022, pp. 1–6. doi:10.1109/ICCSEA54677.2022.9936578.

[79] Ctimd: Cyber threat intelligence enhanced malware detection using api call sequences with parameters, Computers Security 136 (2024) 103518. doi:https://doi.org/10.1016/j.cose.2023.103518.

[80] S. Ilić, M. Gnjatović, I. Tot, B. Jovanović, N. Maček, M. Gavrilović Božović, Going beyond api calls in dynamic malware analysis: A novel dataset, Electronics 13 (17) (2024). doi:10.3390/electronics13173553.
URL https://www.mdpi.com/2079-9292/13/17/3553

[81] S. Yang, Y. Yang, D. Zhao, L. Xu, X. Li, F. Yu, J. Hu, Dynamic malware detection based on supervised contrastive learning, Comput. Electr. Eng. 123 (2025) 110108. doi:10.1016/j.compeleceng.2025.110108.

[82] A. Silberschatz, P. B. Galvin, G. Gagne, Operating System Concepts, 10th Edition, John Wiley & Sons, 2018.

[83] A. Al-Dujaili, A. Huang, E. Hemberg, U.-M. OReilly, Adversarial Deep Learning for Robust Detection of Binary Encoded Malware , in: 2018 IEEE Security and Privacy Workshops (SPW), IEEE Computer Society, Los Alamitos, CA, USA, 2018, pp. 76–82. doi:10.1109/SPW.2018.00020.
URL https://doi.ieeecomputersociety.org/10.1109/SPW.2018.00020

[84] Y. Chen, X. Gao, H. Sun, Memory-based malware detection using side-channel analysis, IEEE Transactions on Information Forensics and Security 17 (2022) 2201–2215.

[85] D. Kirat, G. Vigna, C. Kruegel, Barecloud: Bare-metal analysis-based evasive malware detection, in: Proceedings of the 23rd USENIX Security Symposium (USENIX Security 14), USENIX Association, San Diego, CA, 2014, pp. 287–301, iSBN 978-1-931971-15-7.
URL https://www.usenix.org/conference/usenixsecurity14/technical-sessions/presentation/kirat

[86] R. Canzanese, S. Mancoridis, System call-based detection of malicious processes, 2015, pp. 119–124. doi:10.1109/QRS.2015.26.

[87] Y. Mirsky, T. Doitshman, Y. Elovici, A. Shabtai, Kitsune: An ensemble of autoencoders for online network intrusion detection, in: Proceedings of the 25th Network and Distributed System Security Symposium (NDSS 2018), Internet Society, 2018. doi:10.14722/ndss.2018.23204.
URL https://www.ndss-symposium.org/ndss2018/ndss-2018-programme/#session3a

[88] Y. Jang, M. Woo, D. Brumley, Towards automatic software lineage inference, in: Proceedings of the 22nd USENIX Security Symposium (USENIX Security 13), 2013, pp. 81–96.
URL https://www.usenix.org/conference/usenixsecurity13/technical-sessions/papers/jang

[89] T. Nguyen, M. Orenbach, A. Atamli, Live system call trace reconstruction on linux, Forensic Science International: Digital Investigation 42 (2022) 301398. doi:10.1016/j.fsidi.2022.301398.

[90] V. Mavroeidis, A. Jøsang, Data-driven threat hunting using sysmon, in: Proceedings of the 2nd International Conference on Cryptography, Security and Privacy (ICCSP), ACM, 2021, pp. 82–88, university of Oslo, Norway. doi:10.1145/3199478.3199490.

[91] Cuckoo Sandbox Project, Cuckoo: Automated malware analysis, https://cuckoosandbox.org.

[92] T. S. John, T. Thomas, S. Emmanuel, Graph convolutional networks for android malware detection with system call graphs, in: 2020 Third ISEA Conference on Security and Privacy (ISEA-ISAP), 2020, pp. 162–170. doi:10.1109/ISEA-ISAP49340.2020.235015.

[93] S. D. Nikolopoulos, I. Polenakis, A graph-based model for malware detection and classification using system-call groups, Journal of Computer Virology and Hacking Techniques 13 (1) (2017) 29–46. doi:10.1007/s11416-016-0267-1.
URL https://doi.org/10.1007/s11416-016-0267-1

[94] J. Lopez, L. Babun, H. Aksu, A. S. Uluagac, A survey on function and system call hooking approaches, Journal of Hardware and Systems Security 1 (2) (2017) 114–136. doi:10.1007/s41635-017-0013-2.
URL https://doi.org/10.1007/s41635-017-0013-2

[95] A. Unknown, A survey on function and system call hooking approaches, Tech. rep., Cyber Security Lab, Florida International University (2023).
URL https://csl.fiu.edu/wp-content/uploads/2023/05/hooking_survey.pdf

[96] S. Cesare, Y. Xiang, W. Zhou, Malwise—an effective and efficient classification system for packed and polymorphic malware, IEEE Transactions on Computers 62 (6) (2013) 1193–1206. doi:10.1109/TC.2012.65.

[97] B. Hammi, J. Hachem, A. Rachini, R. Khatoun, H. Aissaoui, Malware detection through windows system call analysis, in: 2024 Ninth International Conference On Mobile And Secure Services (MobiSecServ), Vol. CFP24RAC-ART, 2024, pp. 1–7. doi:10.1109/MobiSecServ63327.2024.10759991.

[98] A. J. Gonzalez, Defining and Calling Functions, Springer International Publishing, Cham, 2020, pp. 65–81. doi:10.1007/978-3-030-50750-3_5.
URL https://doi.org/10.1007/978-3-030-50750-3_5

[99] J. Bartlett, Calling Functions from Libraries, Apress, Berkeley, CA, 2021, pp. 151–163. doi:10.1007/978-1-4842-7437-8_12.
URL https://doi.org/10.1007/978-1-4842-7437-8_12

[100] R. Hodován, D. Vince, Á. Kiss, Fuzzing javascript environment apis with interdependent function calls, in: Integrated Formal Methods, Vol. 11918 of Lecture Notes in Computer Science, Springer International Publishing, Cham, Switzerland, 2019, pp. 212–226. doi:10.1007/978-3-030-34968-4_12.
URL https://doi.org/10.1007/978-3-030-34968-4_12

[101] C. Wang, J. Pang, R. Zhao, W. Fu, X. Liu, Malware detection based on suspicious behavior identification, in: 2009 First International Workshop on Education Technology and Computer Science, Vol. 2, 2009, pp. 198–202. doi:10.1109/ETCS.2009.306.

[102] S. Peisert, M. Bishop, S. Karin, K. Marzullo, Analysis of computer intrusions using sequences of function calls, IEEE Transactions on Dependable and Secure Computing 4 (2) (2007) 137–150. doi:10.1109/TDSC.2007.1003.

[103] X. Ling, L. Wu, W. Deng, Z. Qu, J. Zhang, S. Zhang, T. Ma, B. Wang, C. Wu, S. Ji, Malgraph: Hierarchical graph neural networks for robust windows malware detection, in: IEEE INFOCOM 2022 - IEEE Conference on Computer Communications, 2022, pp. 1998–2007. doi:10.1109/INFOCOM48880.2022.9796786.

[104] O. Kargarnovin, A. M. Sadeghzadeh, R. Jalili, Mal2gcn: A robust malware detection approach using deep graph convolutional networks with non-negative weights, Journal of Computer Virology and Hacking Techniques 19 (1) (2023) 95–111. doi:10.1007/s11416-023-00498-7.
URL https://doi.org/10.1007/s11416-023-00498-7

[105] M. Someya, Y. Otsubo, A. Otsuka, Fcgat: Interpretable malware classification method using function call graph and attention mechanism, in: Proceedings of the Workshop on Binary Analysis Research (BAR) 2023, Internet Society, 2023. doi:10.14722/bar.2023.23005.

[106] M. Hassen, P. K. Chan, Scalable function call graph-based malware classification, in: Proceedings of the Seventh ACM on Conference on Data and Application Security and Privacy, CODASPY '17, Association for Computing Machinery, New York, NY, USA, 2017, p. 239–248. doi:10.1145/3029806.3029824.

[107] M. Tsantekidis, V. Prevelakis, Software system exploration using library call analysis, in: G. Hatzivasilis, S. Ioannidis (Eds.), Model-driven Simulation and Training Environments for Cybersecurity, Springer International Publishing, Cham, 2020, pp. 125–139. doi:10.1007/978-3-030-62433-0_8.

[108] D. Bruening, T. Garnett, S. P. Amarasinghe, An infrastructure for adaptive dynamic optimization, in: Proceedings of the International Symposium on Code Generation and Optimization (CGO), IEEE Computer Society, San Francisco, CA, USA, 2003, pp. 265–276. doi:10.1109/CGO.2003.1191551.
URL https://doi.org/10.1109/CGO.2003.1191551

[109] M. Bach, M. Charney, R. Cohn, E. Demikhovsky, T. Devor, K. M. Hazelwood, A. Jaleel, C.-K. Luk, G. Lyons, H. Patil, A. Tal, Analyzing parallel programs with pin, Computer 43 (3) (2010) 34–41. doi:10.1109/MC.2010.60.
URL https://doi.org/10.1109/MC.2010.60

[110] Z. Liu, R. Wang, N. Japkowicz, H. M. Gomes, B. Peng, W. Zhang, Segdroid: An android malware detection method based on sensitive function call graph learning, Expert Systems with Applications 235 (2024) 121125. doi:https://doi.org/10.1016/j.eswa.2023.121125.

[111] S. Hou, Y. Ye, Y. Song, M. Abdulhayoglu, Make evasion harder: An intelligent android malware detection system, in: Proceedings of the Twenty-Seventh International Joint Conference on Artificial Intelligence, IJCAI-18, International Joint Conferences on Artificial Intelligence Organization, 2018,

pp. 5279–5283. doi:10.24963/ijcai.2018/737.
URL https://doi.org/10.24963/ijcai.2018/737

[112] Aslan, E. Akin, Malware detection method based on file and registry operations using machine learning, Sakarya University Journal of Computer and Information Sciences 5 (2) (2022) 134–146. doi:10.35377/saucis...1049798.
URL https://doi.org/10.35377/saucis...1049798

[113] M. U. Rana, M. Ali Shah, O. Ellahi, Malware persistence and obfuscation: An analysis on concealed strategies, in: 2021 26th International Conference on Automation and Computing (ICAC), 2021, pp. 1–6. doi:10.23919/ICAC50006.2021.9594197.

[114] M. Torres, R. Álvarez, M. Cazorla, A malware detection approach based on feature engineering and behavior analysis, IEEE Access 11 (2023) 105355–105367. doi:10.1109/ACCESS.2023.3319093.

[115] D. M. Jacobsen, procmon: Real-time process monitoring on the cray xc-30, in: Proceedings of the Cray User Group Conference (CUG 2015), Cray User Group, Chicago, IL, USA, 2015.
URL https://cug.org/proceedings/cug2015_proceedings/includes/files/pap175-file1.pdf

[116] N. A. Rosli, W. Yassin, F. M.A, S. R. Selamat, Clustering analysis for malware behavior detection using registry data, International Journal of Advanced Computer Science and Applications 10 (12) (2019). doi:10.14569/IJACSA.2019.0101213.
URL http://dx.doi.org/10.14569/IJACSA.2019.0101213

[117] J. Singh, J. Singh, Detection of malicious software by analyzing the behavioral artifacts using machine learning algorithms, Information and Software Technology 121 (2020) 106273. doi:https://doi.org/10.1016/j.infsof.2020.106273.

[118] D. Trizna, L. Demetrio, B. Biggio, F. Roli, Nebula: Self-attention for dynamic malware analysis, IEEE Transactions on Information Forensics and Security 19 (2024) 6155–6167. doi:10.1109/TIFS.2024.3409083.

[119] D. Escudero García, N. DeCastro-García, Optimal feature configuration for dynamic malware detection, Computers Security 105 (2021) 102250. doi:https://doi.org/10.1016/j.cose.2021.102250.

URL https://www.sciencedirect.com/science/article/pii/S0167404821000742

[120] Y.-T. Huang, Y. S. Sun, M. C. Chen, Tagseq: Malicious behavior discovery using dynamic analysis, PLOS ONE 17 (5) (2022) 1–23. doi:10.1371/journal.pone.0263644.
URL https://doi.org/10.1371/journal.pone.0263644

[121] M. R. H. Iman, P. Chikul, G. Jervan, H. Bahsi, T. Ghasempouri, Anomalous file system activity detection through temporal association rule mining, in: Proceedings of the 9th International Conference on Information Systems Security and Privacy - ICISSP, INSTICC, SciTePress, 2023, pp. 733–740. doi:10.5220/0011805100003405.

[122] S.-H. Han, D. Lee, Kernel-based real-time file access monitoring structure for detecting malware activity, Electronics 11 (12) (2022). doi:10.3390/electronics11121871.
URL https://www.mdpi.com/2079-9292/11/12/1871

[123] S. Sheen, K. A. Asmitha, S. Venkatesan, R-sentry: Deception based ransomware detection using file access patterns, Computers and Electrical Engineering 103 (2022) 108346. doi:https://doi.org/10.1016/j.compeleceng.2022.108346.
URL https://www.sciencedirect.com/science/article/pii/S0045790622005651

[124] A. A. Al-Hashmi, F. A. Ghaleb, A. Al-Marghilani, A. E. Yahya, S. A. Ebad, M. S. Saqib, A. A. Darem, Deep-ensemble and multifaceted behavioral malware variant detection model, IEEE Access 10 (2022) 42762–42777. doi:10.1109/ACCESS.2022.3168794.
URL https://doi.org/10.1109/ACCESS.2022.3168794

[125] Aslan, E. Akin, Malware detection method based on file and registry operations using machine learning, Sakarya University Journal of Computer and Information Sciences 5 (2) (2022) 134–146. doi:10.35377/saucis...1049798.

[126] A. Stewart, H. ManagerKnownDLLs, Dll side-loading: A thorn in the side of the anti-virus industry, 2014.
URL https://api.semanticscholar.org/CorpusID:221741842

[127] A. Verdier, R. Laborde, M. A. Kandi, A. Benzekri, A slahp in the face of dll search order hijacking, in: G. Wang, H. Wang, G. Min, N. Georgalas,

W. Meng (Eds.), Ubiquitous Security, Springer Nature Singapore, Singapore, 2024, pp. 177–190.

[128] T. Rezaei, F. Manavi, A. Hamzeh, A pe header-based method for malware detection using clustering and deep embedding techniques, Journal of Information Security and Applications 60 (2021) 102876. doi:https://doi.org/10.1016/j.jisa.2021.102876.
URL https://www.sciencedirect.com/science/article/pii/S2214212621001046

[129] M. Yousuf, I. Anwer, A. Riasat, K. T. Zia, S. Kim, Windows malware detection based on static analysis with multiple features, PeerJ Computer Science 9 (2023). doi:10.7717/peerj-cs.1319.

[130] D. Gibert, J. Planes, C. Mateu, Q. Le, Fusing feature engineering and deep learning: A case study for malware classification, Expert Systems with Applications 207 (2022) 117957. doi:https://doi.org/10.1016/j.eswa.2022.117957.
URL https://www.sciencedirect.com/science/article/pii/S0957417422011927

[131] R. Chaganti, V. Ravi, T. D. Pham, A multi-view feature fusion approach for effective malware classification using deep learning, Journal of Information Security and Applications 72 (2023) 103402. doi:https://doi.org/10.1016/j.jisa.2022.103402.
URL https://www.sciencedirect.com/science/article/pii/S2214212622002460

[132] K. Khalda, D. K. Wibowo, Analisis perilaku malware menggunakan pendekatan analisis statis dan dinamis, Jurnal Sains, Nalar, dan Aplikasi Teknologi Informasi 4 (1) (2025) 1–8. doi:10.20885/snati.v4.i1.1.
URL https://journal.uii.ac.id/jurnalsnati/article/view/37335

[133] H. H. Al-Khshali, M. Ilyas, Impact of portable executable header features on malware detection accuracy, Computers, Materials & Continua 74 (1) (2023) 153–178. doi:10.32604/cmc.2023.032182.
URL https://doi.org/10.32604/cmc.2023.032182

[134] T. Q. Dam, N. T. Nguyen, T. V. Le, T. D. Le, S. Uwizeyemungu, T. Le-Dinh, Visualizing portable executable headers for ransomware detection: A deep learning-based approach, Journal of Universal Computer Science 30 (2) (2024) 262–286. doi:10.3897/jucs.104901.
URL https://doi.org/10.3897/jucs.104901

[135] S. Jamalpur, Y. S. Navya, P. Raja, G. Tagore, G. R. K. Rao, Dynamic malware analysis using cuckoo sandbox, in: 2018 Second International Conference on Inventive Communication and Computational Technologies (ICICCT), 2018, pp. 1056–1060. doi:10.1109/ICICCT.2018.8473346.

[136] J. Singh, J. Singh, Detection of malicious software by analyzing the behavioral artifacts using machine learning algorithms, Information and Software Technology 121 (2020) 106273. doi:https://doi.org/10.1016/j.infsof.2020.106273.
URL https://www.sciencedirect.com/science/article/pii/S0950584920300239

[137] A. Cannarile, V. Dentamaro, S. Galantucci, A. Iannacone, D. Impedovo, G. Pirlo, Comparing deep learning and shallow learning techniques for api calls malware prediction: A study, Applied Sciences 12 (3) (2022). doi:10.3390/app12031645.
URL https://www.mdpi.com/2076-3417/12/3/1645

[138] W. Matsuda, M. Fujimoto, T. Mitsunaga, Real-time detection system against malicious tools by monitoring dll on client computers, in: 2019 IEEE Conference on Application, Information and Network Security (AINS), 2019, pp. 36–41. doi:10.1109/AINS47559.2019.8968697.

[139] A. Kumar, K. P. Singh, J. Wasim, M. Chaudhary, Multiclassification of malware using dll, in: 2024 International Conference on Emerging Technologies and Innovation for Sustainability (EmergIN), 2024, pp. 585–588. doi:10.1109/EmergIN63207.2024.10961686.

[140] S. Gulmez, A. G. Kakisim, I. Sogukpinar, Analysis of the dynamic features on ransomware detection using deep learning-based methods, in: 2023 11th International Symposium on Digital Forensics and Security (ISDFS), 2023, pp. 1–6. doi:10.1109/ISDFS58141.2023.10131862.

[141] S. Gulmez, A. Gorgulu Kakisim, I. Sogukpinar, Xran: Explainable deep learning-based ransomware detection using dynamic analysis, Computers Security 139 (2024) 103703. doi:https://doi.org/10.1016/j.cose.2024.103703.
URL https://www.sciencedirect.com/science/article/pii/S016740482400004X

[142] W. B. Andreopoulos, Malware Detection with Sequence-Based Machine Learning and Deep Learning, Springer International Publishing, Cham, 2021,

pp. 53–70. doi:10.1007/978-3-030-62582-5_2.
URL https://doi.org/10.1007/978-3-030-62582-5_2

[143] H. Zhang, B. Li, W. Li, L. Zhu, C. Chang, S. Yu, Mrcif: A memory-reverse-based code injection forensics algorithm, Applied Sciences 13 (4) (2023). doi:10.3390/app13042478.
URL https://www.mdpi.com/2076-3417/13/4/2478

[144] B. Panda, S. N. Tripathy, Detection of anomalous in-memory process based on dll sequence, International Journal of Advanced Computer Science and Applications 11 (10) (2020). doi:10.14569/IJACSA.2020.0111025.
URL http://dx.doi.org/10.14569/IJACSA.2020.0111025

[145] A. F. Muhtadi, A. Almaarif, Analysis of malware impact on network traffic using behavior-based detection technique, International Journal of Advances in Data and Information Systems 1 (1) (2020) 17–25. doi:https://doi.org/10.25008/ijadis.v1i1.14.

[146] B. N. Raj, U. R. K.B, V. Srujan, S. Rajagopal, Network-centric malware detection using dynamic packet analysis and hybrid architectures, in: 2025 International Conference on Intelligent Systems and Computational Networks (ICISCN), IEEE, 2025, pp. 1–5. doi:10.1109/ICISCN64258.2025.10934187.

[147] A. Pasquini, R. Vasa, I. Logothetis, H. Habibi Gharakheili, A. Chambers, M. Tran, Robust and lightweight modeling of iot network behaviors from raw traffic packets, IEEE Transactions on Machine Learning in Communications and Networking 3 (2025) 98–116. doi:10.1109/TMLCN.2024.3517613.

[148] S. M. Z. Javed, M. F. Amjad, Enhancing malware classification through dynamic behavioral analysis, siem integration, and deep learning, in: 2024 International Seminar on Intelligent Technology and Its Applications (ISITIA), 2024, pp. 663–668. doi:10.1109/ISITIA63062.2024.10667741.

[149] K. Rana, S. Gupta, G. Kaur, A. L. Yadav, Malware detection in network traffic using machine learning, in: 2024 3rd International Conference on Applied Artificial Intelligence and Computing (ICAAIC), 2024, pp. 358–362. doi:10.1109/ICAAIC60222.2024.10575355.

[150] C. Huang, R. Wang, J. Zhao, L. Zhang, Malicious network traffic detection method based on traffic behavior characteristics and machine learning, in: 2023 IEEE 6th International Conference on Automation, Elec-

tronics and Electrical Engineering (AUTEEE), 2023, pp. 167–172. doi:10.1109/AUTEEE60196.2023.10407590.

[151] A. H. Omopintemi, I. Ghafir, S. Eltanani, S. Kabir, M. Lefoane, Machine learning for malware detection in network traffic, in: Proceedings of the 7th International Conference on Future Networks and Distributed Systems, ICFNDS '23, Association for Computing Machinery, New York, NY, USA, 2024, p. 605–610. doi:10.1145/3644713.3644804.

[152] G. Kale, G. E. Bostancı, F. V. Çelebi, Evolutionary feature selection for machine learning based malware classification, Engineering Science and Technology, an International Journal 56 (2024) 101762. doi:https://doi.org/10.1016/j.jestch.2024.101762.
URL https://www.sciencedirect.com/science/article/pii/S2215098624001484

[153] B. Panda, S. N. Tripathy, Detection of anomalous in-memory process based on dll sequence, International Journal of Advanced Computer Science and Applications 11 (10) (2020). doi:10.14569/IJACSA.2020.0111025.
URL http://dx.doi.org/10.14569/IJACSA.2020.0111025

[154] K. S. ManiArasuSekar, P. Swaminathan, R. Murali, G. K. Ratan, S. V. Siva, Optimal feature selection for non-network malware classification, in: 2020 International Conference on Inventive Computation Technologies (ICICT), 2020, pp. 82–87. doi:10.1109/ICICT48043.2020.9112437.

[155] W. Aman, A framework for analysis and comparison of dynamic malware analysis tools, International Journal of Network Security & Its Applications 6 (5) (2014) 63–74. doi:10.5121/ijnsa.2014.6505.

[156] C. Willems, T. Holz, F. C. Freiling, Toward automated dynamic malware analysis using cwsandbox, IEEE Security & Privacy 5 (2) (2007) 32–39. doi:10.1109/MSP.2007.45.
URL https://doi.org/10.1109/MSP.2007.45

[157] M. Guven, Dynamic malware analysis using a sandbox environment, network traffic logs, and artificial intelligence., International Journal of Computational and Experimental Science and Engineering (2024). doi:10.22399/ijcesen.460.

[158] M. Russinovich, Process explorer, https://learn.microsoft.com/en-us/sysinternals/downloads/process-explorer, microsoft Sysinternals (2009).

[159] R. Sihwail, K. Omar, K. A. Z. Ariffin, An effective memory analysis for malware detection and classification, Computer Modeling in Engineering & Sciences (2020).
URL https://www.researchgate.net/figure/Malware-DLL-injection-technique_fig2_348977684

[160] R. P. Pokhrel, Behavioral analysis of malware using sandboxing techniques, International Journal of Science and Research Archive (2025). doi:10.30574/ijsra.2025.15.3.1781.

[161] S. Ilić, M. Gnjatović, B. Popović, N. Maček, A pilot comparative analysis of the cuckoo and drakvuf sandboxes: An end-user perspective, Vojnotehnicki glasnik (2022). doi:10.5937/vojtehg70-36196.

[162] Preeti, A. Agrawal, A comparative analysis of open source automated malware tools, 2022 9th International Conference on Computing for Sustainable Global Development (INDIACom) (2022) 226–230doi:10.23919/indiacom54597.2022.9763227.

[163] A. Mills, P. Legg, Investigating anti-evasion malware triggers using automated sandbox reconfiguration techniques, J. Cybersecur. Priv. 1 (2020) 19–39. doi:10.20944/preprints202010.0305.v1.

[164] I. Vurdelja, I. Blazic, D. Bojic, D. Draskovic, A framework for automated dynamic malware analysis for linux, in: 2020 28th Telecommunications Forum (TELFOR), IEEE, 2020, p. 1–4. doi:10.1109/telfor51502.2020.9306520.
URL http://dx.doi.org/10.1109/telfor51502.2020.9306520

[165] U. E. Essien, S. I. Ele, Cuckoo sandbox and process monitor (procmon) performance evaluation in large-scale malware detection and analysis, British Journal of Computer, Networking and Information Technology (2024). doi:10.52589/bjcnit-fcedoomy.

[166] F. Alhaidari, N. A. Shaib, M. Alsafi, H. Alharbi, M. Alawami, R. Aljindan, A. ur Rahman, R. Zagrouba, Zevigilante: Detecting zero-day malware using machine learning and sandboxing analysis techniques, Computational Intelligence and Neuroscience 2022 (2022). doi:10.1155/2022/1615528.

[167] A. R. Melvin, G. J. W. Kathrine, A quest for best: A detailed comparison between drakvuf-vmi-based and cuckoo sandbox-based technique for dynamic malware analysis (2020) 275–290doi:10.1007/978-981-15-5285-4_27.

[168] M. Z. Muzahid, M. B. Akram, A. Alamgir, Analysis of agent-based and agent-less sandboxing for dynamic malware analysis (2020) 72–84doi:10.1007/978-3-030-52856-0_6.

[169] A. Dinaburg, P. Royal, M. Sharif, W. Lee, Ether: malware analysis via hardware virtualization extensions, in: Proceedings of the 15th ACM conference on Computer and communications security, 2008, pp. 51–62.

[170] G. E. Oki, E. W. Sadiq, Virtual machines vs. containerized environments: A comparative study for malware analysis, Saudi Journal of Engineering and Technology 10 (04) (2025) 206–210. doi:10.36348/sjet.2025.v10i04.011.
URL https://doi.org/10.36348/sjet.2025.v10i04.011

[171] S. Liu, P. Feng, S. Wang, K. Sun, J. Cao, et al., Enhancing malware analysis sandboxes with emulated user behavior, Computers & Security 115 (2022) 102613. doi:10.1016/j.cose.2022.102613.
URL https://doi.org/10.1016/j.cose.2022.102613

[172] R. Saikumar, S. Sahay, M. S. Gaur, Malware detection using dynamic api call sequence and machine learning, Procedia Computer Science 167 (2020) 2061–2069. doi:10.1016/j.procs.2020.03.357.
URL https://doi.org/10.1016/j.procs.2020.03.357

[173] M. Sysinternals, Process monitor (procmon), https://learn.microsoft.com/en-us/sysinternals/downloads/procmon, accessed: 2025-06-06.

[174] Frida Project, Frida: Dynamic instrumentation toolkit, https://frida.re.

[175] C.-K. Luk, R. Cohn, R. Muth, H. Patil, A. Klauser, G. Lowney, S. Wallace, V. J. Reddi, K. Hazelwood, Pin: Building customized program analysis tools with dynamic instrumentation, in: Proceedings of the 2005 ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI), ACM, Chicago, IL, USA, 2005, pp. 190–200. doi:10.1145/1064978.1065034.
URL https://doi.org/10.1145/1064978.1065034

[176] Hex-Rays, Ida pro disassembler, https://hex-rays.com/ida-pro/.

[177] National Security Agency, Ghidra software reverse engineering framework, https://ghidra-sre.org.

[178] K. Rieck, P. Trinius, C. Willems, T. Holz, Automatic analysis of malware behavior using machine learning, Journal of Computer Security 19 (4) (2011)

639–668. doi:10.3233/JCS-2010-0410.
URL https://doi.org/10.3233/JCS-2010-0410

[179] A. Muzaffar, H. R. Hassen, H. Zantout, M. A. Lones, Droiddissector: A static and dynamic analysis tool for android malware detection, in: Proceedings of the International Conference on Applied CyberSecurity (ACS) 2023, Vol. 760 of Lecture Notes in Networks and Systems, Springer Nature Switzerland, 2023, pp. 3–9. doi:10.1007/978-3-031-40598-3_1.

[180] J. Bania, Hasherezade, Pe-sieve – a tool for hunting malware implants, https://github.com/hasherezade/pe-sieve, accessed: 2025-06-06 (2021).

[181] M. Russinovich, Sysmon – system monitor, https://learn.microsoft.com/en-us/sysinternals/downloads/sysmon, microsoft Sysinternals (2014).

[182] G. Combs, Wireshark: Network protocol analyzer, https://www.wireshark.org/, accessed: 2025-06-06 (2010).

[183] T. Siko, Fakenet-ng – next generation dynamic network analysis tool, https://github.com/mandiant/flare-fakenet-ng, accessed: 2025-06-06 (2016).

[184] Seabreg, Regshot – registry compare utility (github repository), https://github.com/Seabreg/Regshot, latest commit active; version 1.9.0 (Feb 2, 2013) (2013). doi:GitHub.

[185] Regshot 2.1 registry compare utility, https://regshot.informer.com/, version 2.1.0.17 (Mar. 29 2025).

[186] M. Russinovich, Autoruns for windows, https://learn.microsoft.com/en-us/sysinternals/downloads/autoruns, accessed: 2025-06-06 (2004).

[187] X. Wan, Y. Sun, Y. Wang, M. Xia, Android malware detection method based on machine learning, in: C. Xu, H. Pan, C. Yu, Q. Han, X. Song (Eds.), Infrastructure for Data Science, Communications in Computer and Information Science (CCIS), Springer Singapore, Macao, SAR, China, 2024, pp. 3–18. doi:10.1007/978-981-97-8749-4_1.
URL https://doi.org/10.1007/978-981-97-8749-4_1

[188] M. D'Onghia, M. Salvadore, B. M. Nespoli, M. Carminati, M. Polino, S. Zanero, Apìcula: Static detection of api calls in generic streams of bytes, Computers & Security 119 (2022) 102775. doi:10.1016/j.cose.2022.102775.

[189] G. Hunt, D. Brubacher, et al., Detours: Binary interception of win32 functions, in: Microsoft Research Technical Report, 1999.
URL https://www.microsoft.com/en-us/research/publication/detours-binary-interception-of-win32-functions/

[190] S. Choi, T. Chang, C. Kim, Y. Park, x64unpack: Hybrid emulation unpacker for 64-bit windows environments and detailed analysis results on vmprotect 3.4, IEEE Access 8 (2020) 127939–127953. doi:10.1109/ACCESS.2020.3008900.

[191] M. Gebai, M. R. Dagenais, Survey and analysis of kernel and userspace tracers on linux: Design, implementation, and overhead, ACM Computing Surveys 51 (2) (2018) 26:1–26:33. doi:10.1145/3158644.

[192] I. Shakhsheer, M. Abdallah, I. Mashaqi, A. Awad, Automated forensic analysis following memory content using volatility framework, in: 2023 International Conference on Innovation and Intelligence for Informatics, Computing, and Technologies (3ICT), IEEE, 2023, pp. 369–374. doi:10.1109/3ICT60104.2023.10391513.
URL https://doi.org/10.1109/3ICT60104.2023.10391513

[193] R. Palutke, F. Block, P. Reichenberger, D. Stripeika, Hiding process memory via anti-forensic techniques, Forensic Science International: Digital Investigation 33 (2020). doi:10.1016/j.fsidi.2020.301012.

[194] J. Ottmann, F. Breitinger, F. Freiling, An experimental assessment of inconsistencies in memory forensics, ACM Transactions on Privacy and Security 27 (2023) 1–29. doi:10.1145/3628600.

[195] S. N. Maulina, N. D. W. Cahyani, E. M. Jadied, Analysis of the effect of vsm on the memory acquisition process using the dynamic analysis method, JIPI (Jurnal Ilmiah Penelitian dan Pembelajaran Informatika) 8 (2) (2023) 638–646. doi:10.29100/jipi.v8i2.3745.

[196] P. Mishra, I. Verma, S. Gupta, Kvminspector: Kvm based introspection approach to detect malware in cloud environment, Journal of Information Security and Applications 51 (2020) 102460. doi:https://doi.org/10.1016/j.jisa.2020.102460.

[197] Which open-source ids? snort, suricata or zeek, Computer Networks 213 (2022) 109116. doi:https://doi.org/10.1016/j.comnet.2022.109116.

[198] M. Elam, D. Mink, S. S. Bagui, R. Plenkers, S. C. Bagui, Introducing uwf-zeekdata24: An enterprise mitre attck labeled network attack traffic dataset for machine learning/ai, Data 10 (5) (2025).
URL https://www.mdpi.com/2306-5729/10/5/59

[199] M. H. Qutqut, A. Ahmed, M. K. Taqi, J. Abimanyu, E. T. Ajes, F. A. Alhaj, A comparative evaluation of snort and suricata for detecting data exfiltration tunnels in cloud environments, Journal of Cybersecurity and Privacy 6 (1) (2026) 17. doi:10.3390/jcp6010017.

[200] L. F. Sikos, Packet analysis for network forensics: A comprehensive survey, Forensic Science International: Digital Investigation 32 (2020) 200892. doi:10.1016/j.fsidi.2019.200892.

[201] P. Boyanov, Investigating the network traffic using the command-line packet sniffer tcpdump in kali linux, Journal Scientific and Applied Research 25 (1) (2023) 31–44. doi:10.46687/jsar.v25i1.378.
URL https://doi.org/10.46687/jsar.v25i1.378

[202] P. Matoušek, I. Burgetová, O. Ryšavý, M. Victor, On reliability of ja3 hashes for fingerprinting mobile applications, in: G. Quirchmayr, J. Basl, I. You, L. Xu, E. Weippl (Eds.), ICT Systems Security and Privacy Protection, Vol. 12703 of Lecture Notes in Computer Science, Springer, 2021, pp. 224–238. doi:10.1007/978-3-030-78120-0_15.

[203] C. Kurniawan, A. Triayudi, File integrity monitoring as a method for detecting and preventing web defacement attacks, JOIN: Jurnal Online Informatika 9 (2) (2024) 276–285. doi:10.15575/join.v9i2.1326.

[204] A. Afianian, S. Niksefat, B. Sadeghiyan, D. Baptiste, Malware dynamic analysis evasion techniques: A survey, ACM Comput. Surv. 52 (6) (Nov. 2019). doi:10.1145/3365001.
URL https://doi.org/10.1145/3365001

[205] N. Galloro, M. Polino, M. Carminati, A. Continella, S. Zanero, A systematical and longitudinal study of evasive behaviors in windows malware, Computers Security 113 (2022) 102550. doi:https://doi.org/10.1016/j.cose.2021.102550.
URL https://www.sciencedirect.com/science/article/pii/S0167404821003746

[206] R. S. Leon, M. Kiperberg, A. A. Leon Zabag, N. J. Zaidenberg, Hypervisor-assisted dynamic malware analysis, Cybersecurity 4 (1) (2021) 19. doi:

10.1186/s42400-021-00083-9.
URL https://doi.org/10.1186/s42400-021-00083-9

Title page with author details

"Dynamic Features for Robust Malware Detection: A Systematic Review, Taxonomy, and Practical Analysis Framework"

Monday Onoja[a,*], Peter Anthony[a], Zekeri Adams[a], Kefas Rimamnuskeb Galadima[b], Martin Homola[a], Štefan Balogh[b], Ján Kľuka[a], Roderik Ploszek[b], Temidayo Oluwatosin Omotehinwa[c], Abayomi Joshua Jegede[d]

[a]Department of Applied Informatics, Faculty of Mathematics, Physics and Informatics, Comenius University, Mlynská dolina, Bratislava, Slovakia

[b]Institute of Computer Science and Mathematics, Faculty of Electrical Engineering and Information Technology, Slovak University of Technology, Ilkovičova 3, Bratislava, Slovakia

[c]Department of Mathematics and Computer Science, Federal University of Health Sciences, , Otukpo, Benue State, Nigeria

[d]Department of Computer Sciences, Edo State University, Km7, Auchi-Abuja Road, Iyamho-Uzairue, Edo State , Nigeria

∗Corresponding author.
Email address: monday.onoja@fmph.uniba.sk
(Monday Onoja: https://orcid.org/0000-0003-2119-170X)